Teoria dos Grafos

Bruno Pereira dos Santos

Aula baseada nos cursos de outros docentes da UFMG (Prof. Antonio Alfredo Ferreira Loureiro), USP (Prof. Paulo Feofiloff), UFBA, livros, artigos e outros.

07 - Travessia em Árvores

Traversal Algorithms

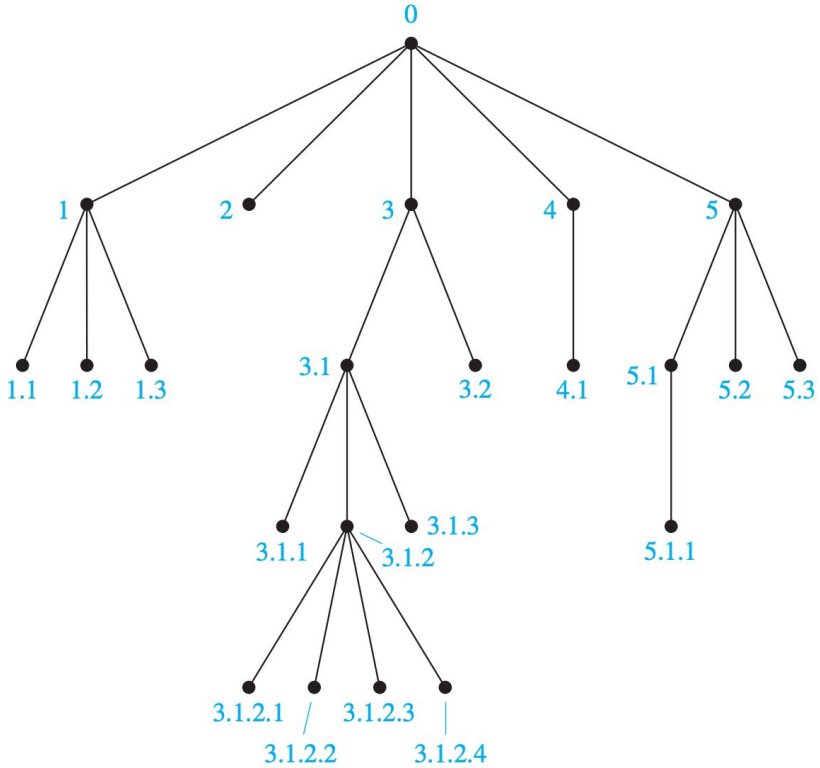
Travessia em Árvores

- Árvores enraizadas ordenadas são usadas para armazenar informações
- Precisamos de algoritmos para visitar todos os vértices
- Aplicações:
  - Representação de expressões (aritméticas, por exemplo)
  - Avaliação de expressões
  - Diferentes ordenações dos vértices → diferentes interpretações

Motivação: *Universal Address System*

- Definição: O *universal address system* é um método de rotular os vértices de uma árvore enraizada ordenada de forma que cada rótulo represente o caminho da raiz até o vértice.
- Ideia principal:
  - Ordenar os vértices usando rótulos hierárquicos
- Procedimento:
  - Rotule a raiz como 0
  - Rotule seus k filhos (nível 1) da esquerda para direita:
    - 1, 2, ..., k
  - Para cada vértice v com rótulo A:
    - Seus filhos recebem:
      - $A.1$, $A.2$, ..., $A.k$

Motivação: *Universal Address System* (1)

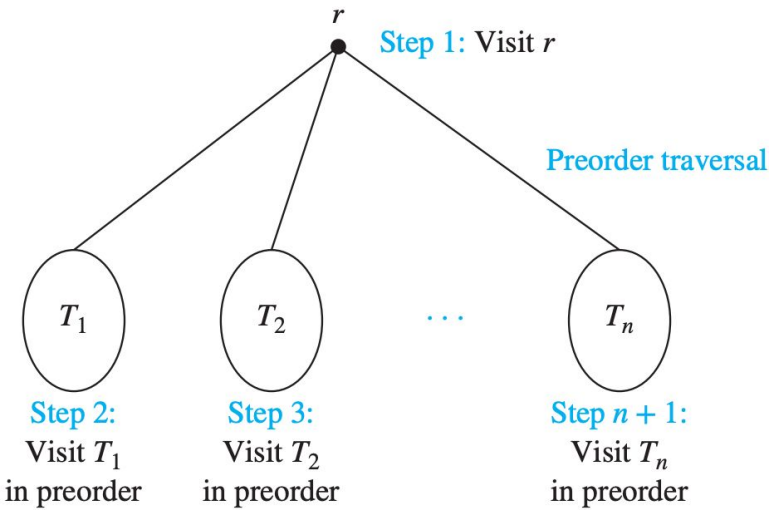


Motivação: *Universal Address System* (2)

- Cada vértice recebe um rótulo do tipo: $x_1.x_2. \ldots .x_n$
  - Representa o caminho da raiz até o vértice
- Ordenação Lexicográfica dos Vértices:
  - Comparação lexicográfica (como palavras)
  - Podemos ordenar totalmente os vértices usando os rótulos do *universal address system*.
  - Sejam: $\boldsymbol{x} = x_1.x_2. \ldots .x_{\boldsymbol{n}}$ e $\boldsymbol{y} = y_1.y_2. \ldots .y_{\boldsymbol{m}}$
    - Regra de comparação: dizemos que x < y se:
      1. Existe i tal que: $x_1 = y_1, \ldots, x_{i-1} = y_{i-1}$
      2. e: $x_i < y_i$
    - Se x é prefixo de y (ou seja, $n < m$ e todos iguais até n): $x < y$
  - Exemplo: 0 < 1 < 1.1 < 1.2 < 2 < 3 < 3.1 < ...
- Isso define uma ordem total dos vértices

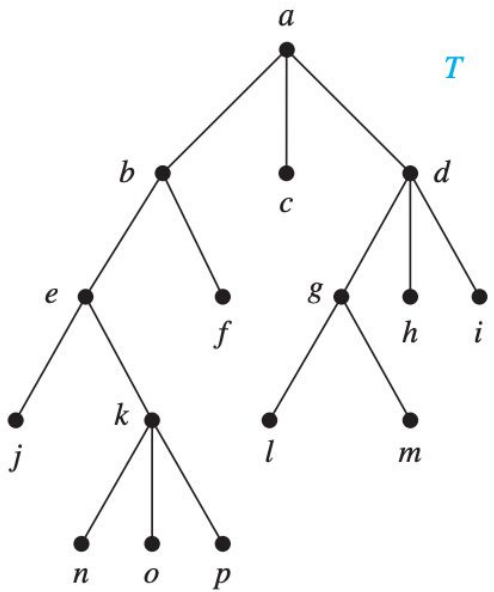
Traversal Algorithms

- Definição: Algoritmos de travessia são procedimentos para visitar sistematicamente todos os vértices de uma árvore enraizada ordenada.
- Principais tipos:
  - *Preorder* (pré-ordem)
  - *Inorder* (em-ordem)
  - *Postorder* (pós-ordem)
- Veja-1 (link)
- Veja-2 (link)



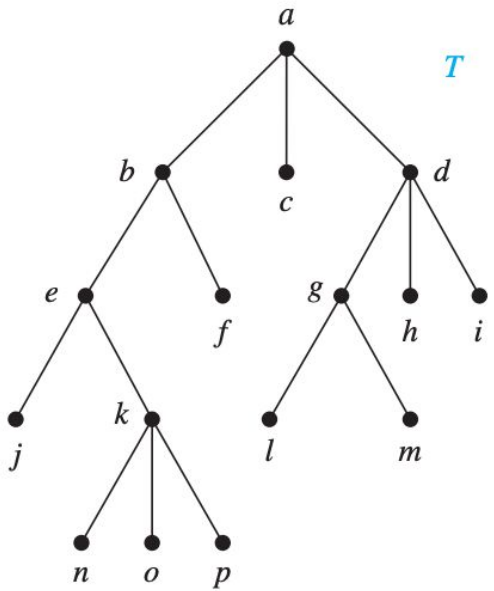
Preorder Traversal

- <u>Definição:</u> seja T uma árvore enraizada ordenada com raiz r.
  - Se T contém apenas r: preorder(T) = r
  - Caso contrário:
    - Sejam $T_1, T_2, \ldots, T_n$ as subárvores de r (da esquerda para a direita)
- Algoritmo
  1. Visite a raiz r
  2. Percorra recursivamente:
     - T_1 em *preorder*
     - T_2 em *preorder*
     - ...
     - T_n em *preorder*



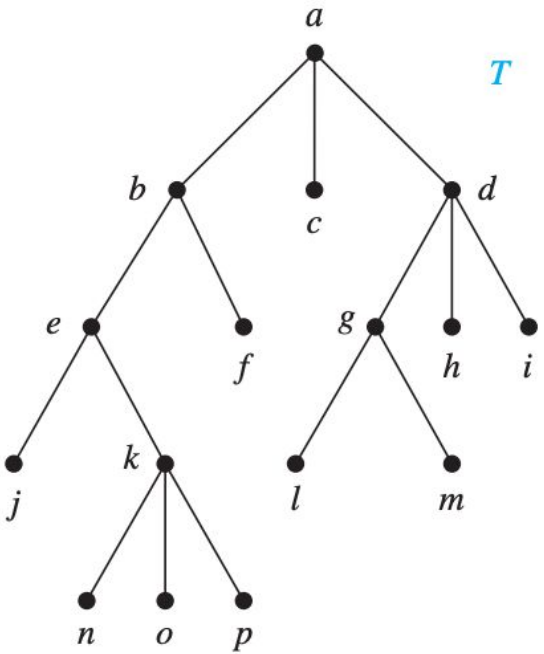
Preorder Traversal (1)

- Problema: Em qual ordem o algoritmo de preorder traversal visita os vértices da árvore T ?
  - Regra do preorder:
    - Visita a raiz
    - Depois visita as subárvores da esquerda para a direita

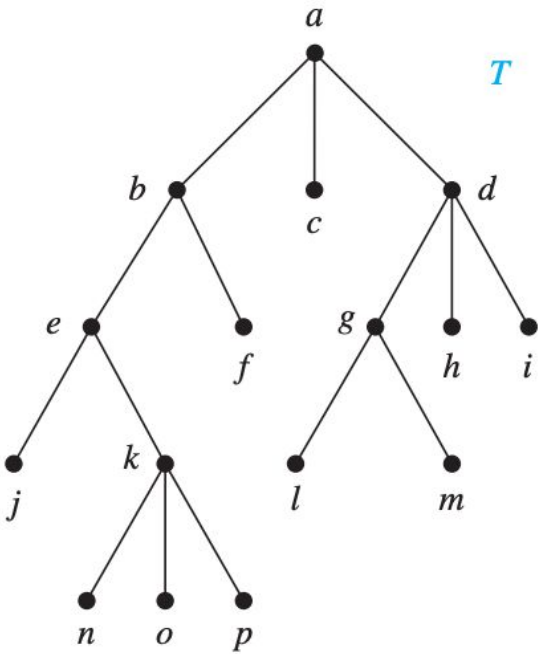


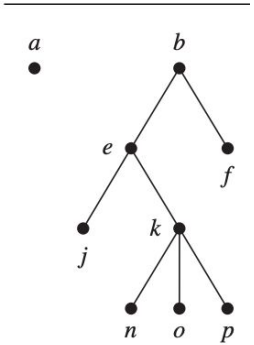


Preorder Traversal (2)

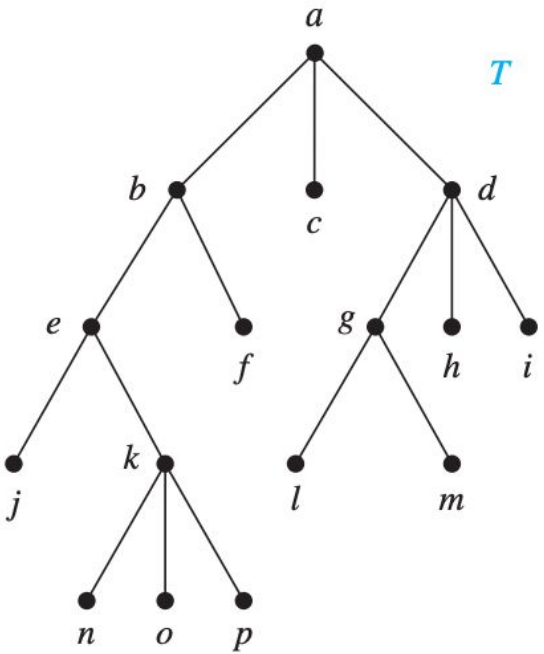


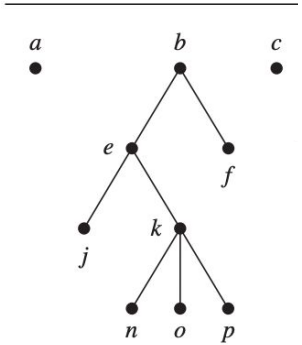
Preorder Traversal (3)



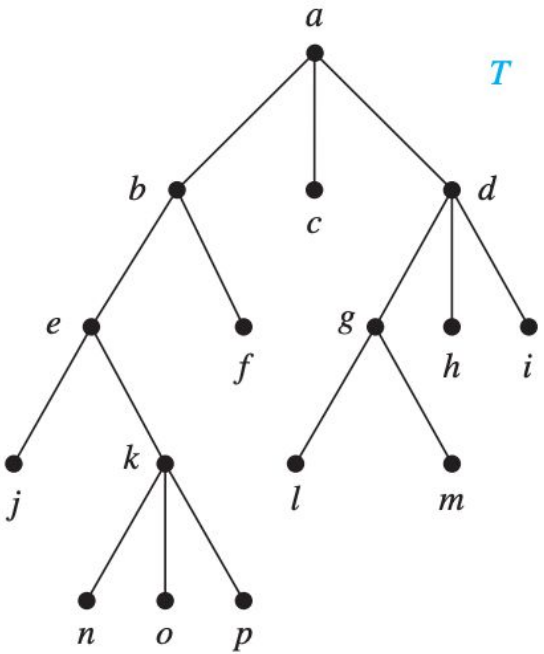


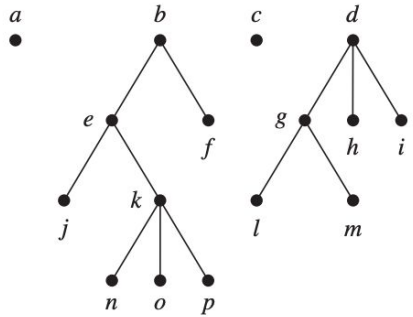
Preorder Traversal (4)



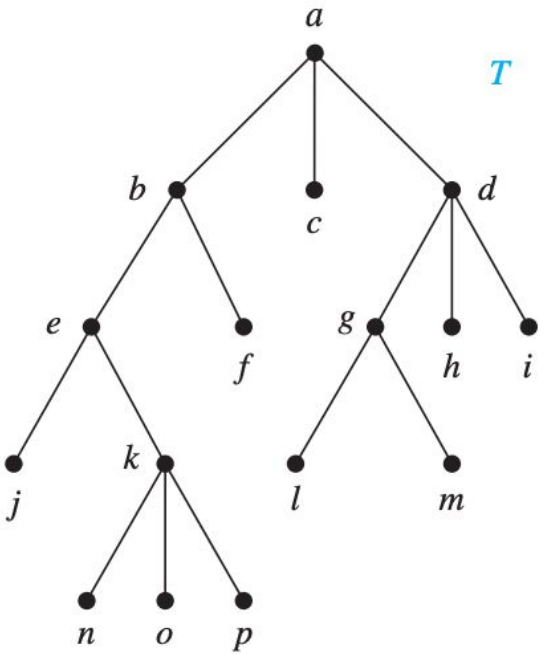


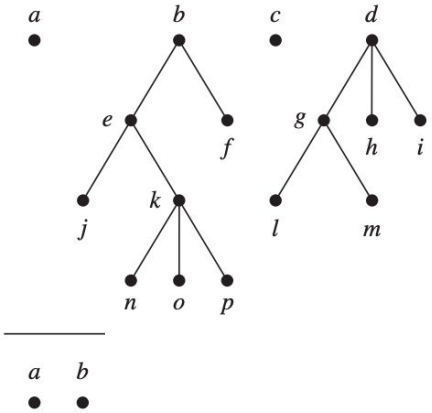
Preorder Traversal (5)



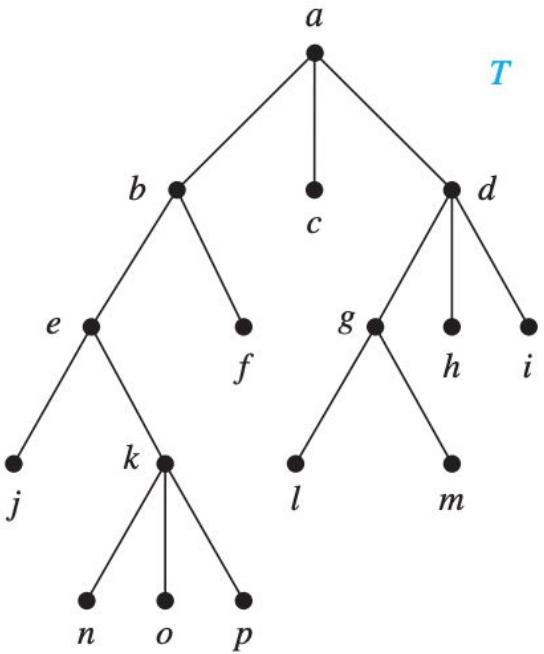


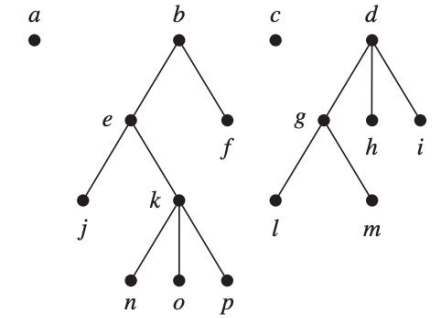
Preorder Traversal (6)

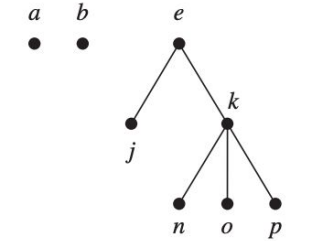




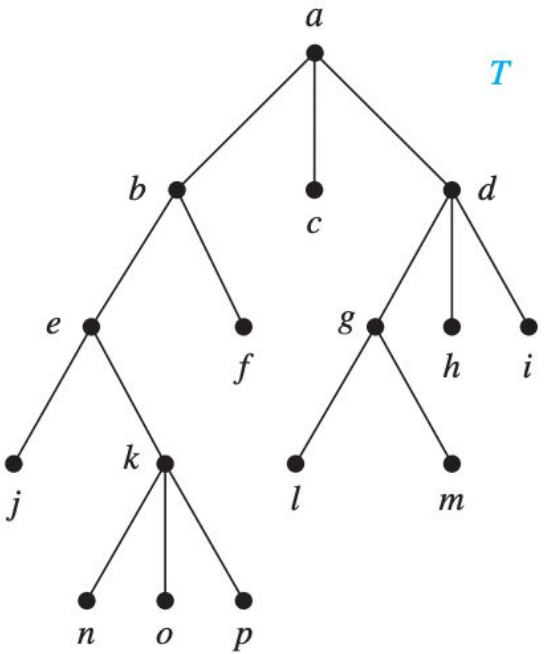
Preorder Traversal (7)

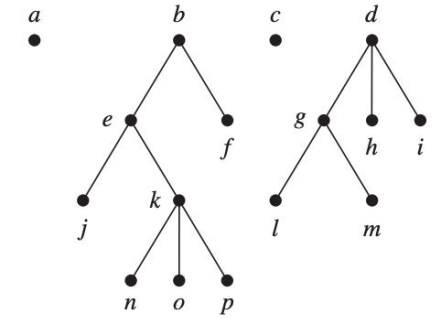


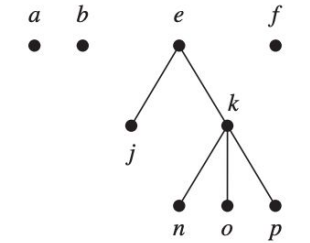




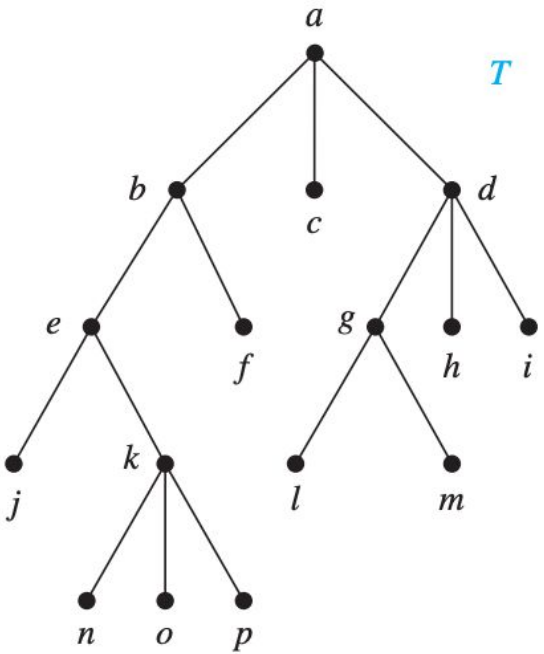
Preorder Traversal (8)

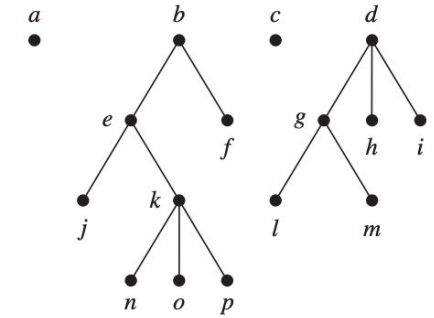


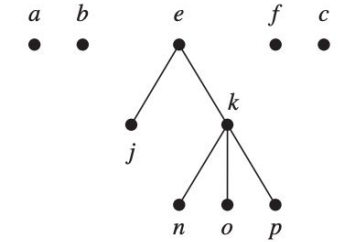




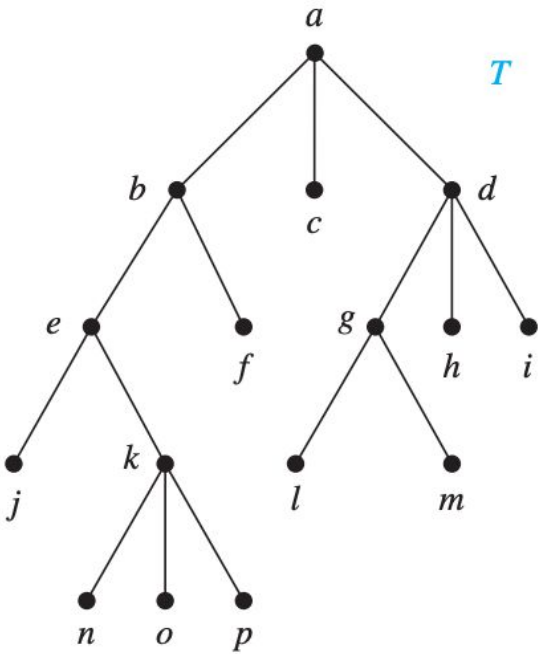
Preorder Traversal (9)

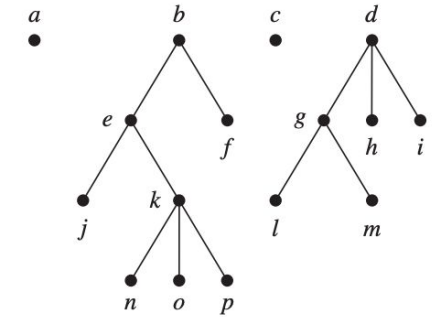


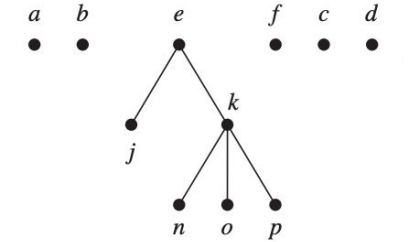




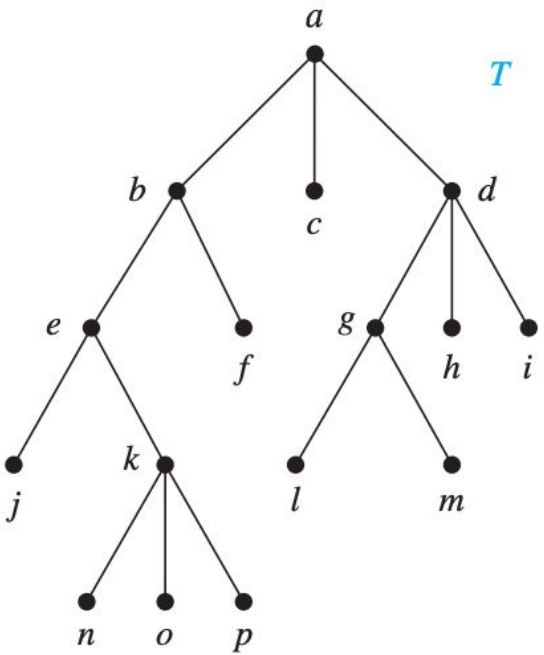
Preorder Traversal (10)

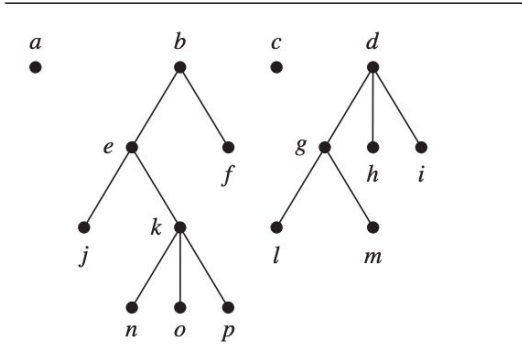


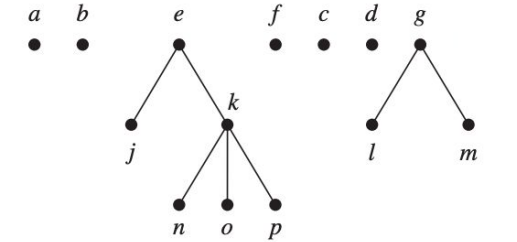




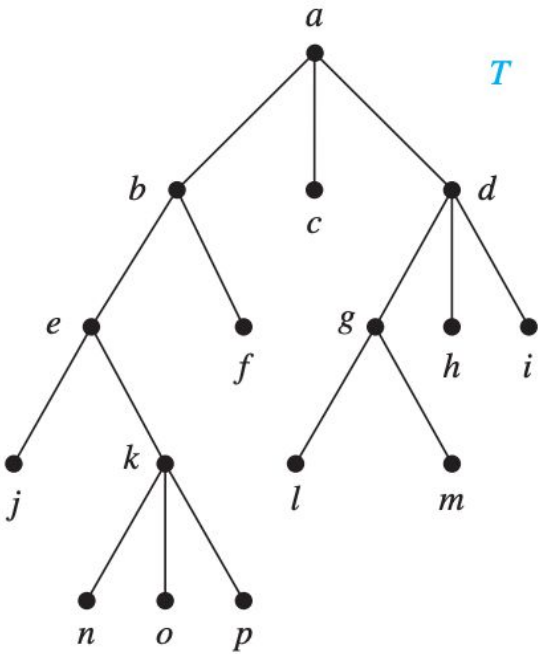
Preorder Traversal (11)

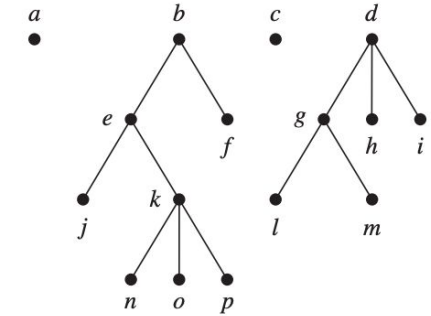


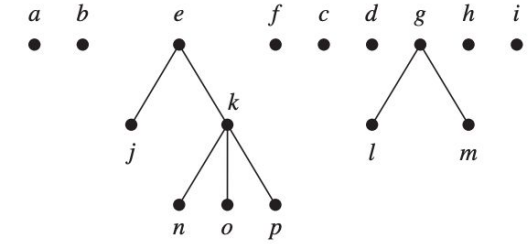


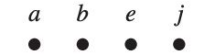


Preorder Traversal (12)

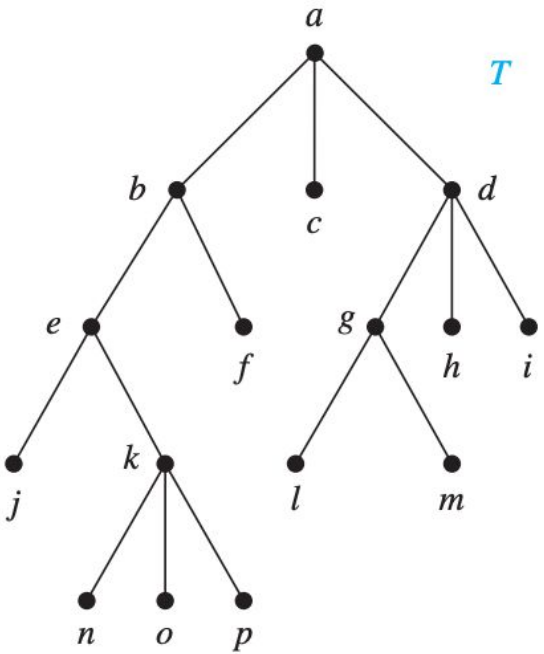


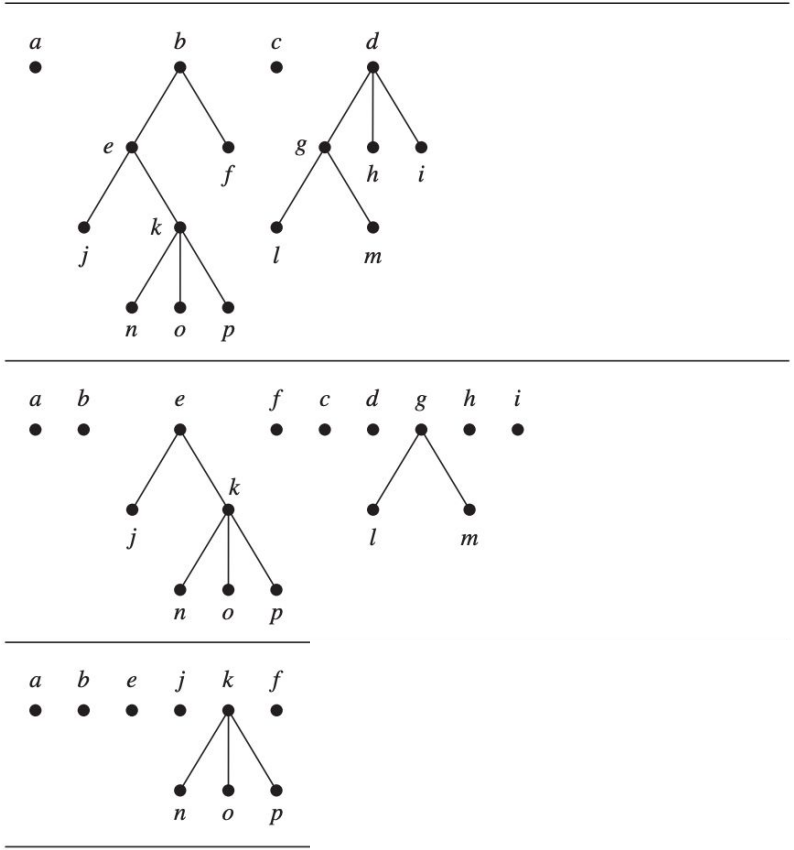




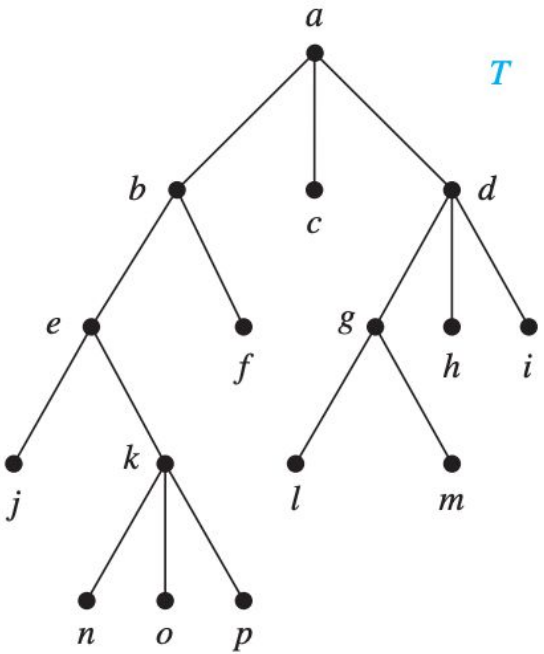


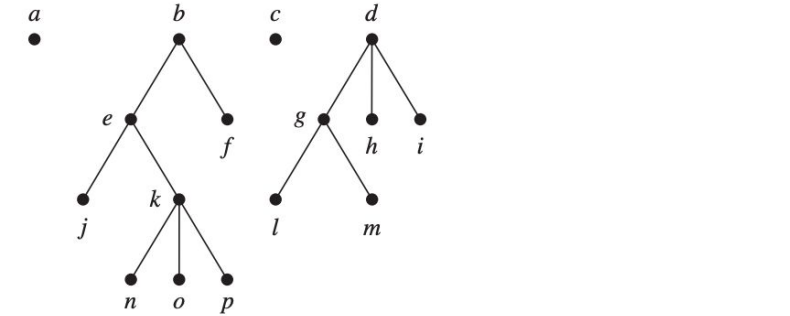
Preorder Traversal (13)

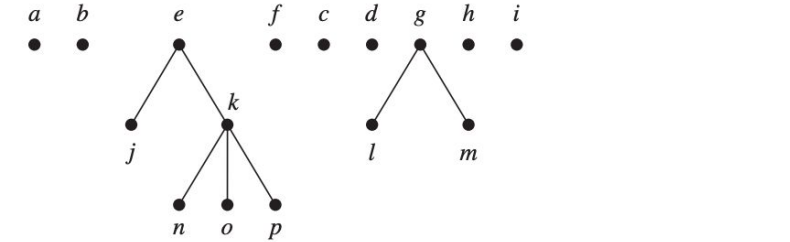


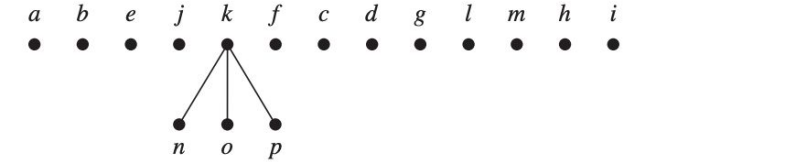


Preorder Traversal (14)

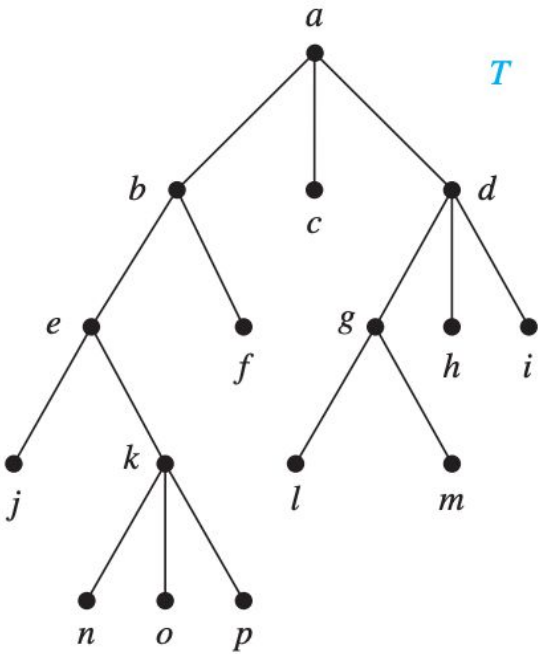


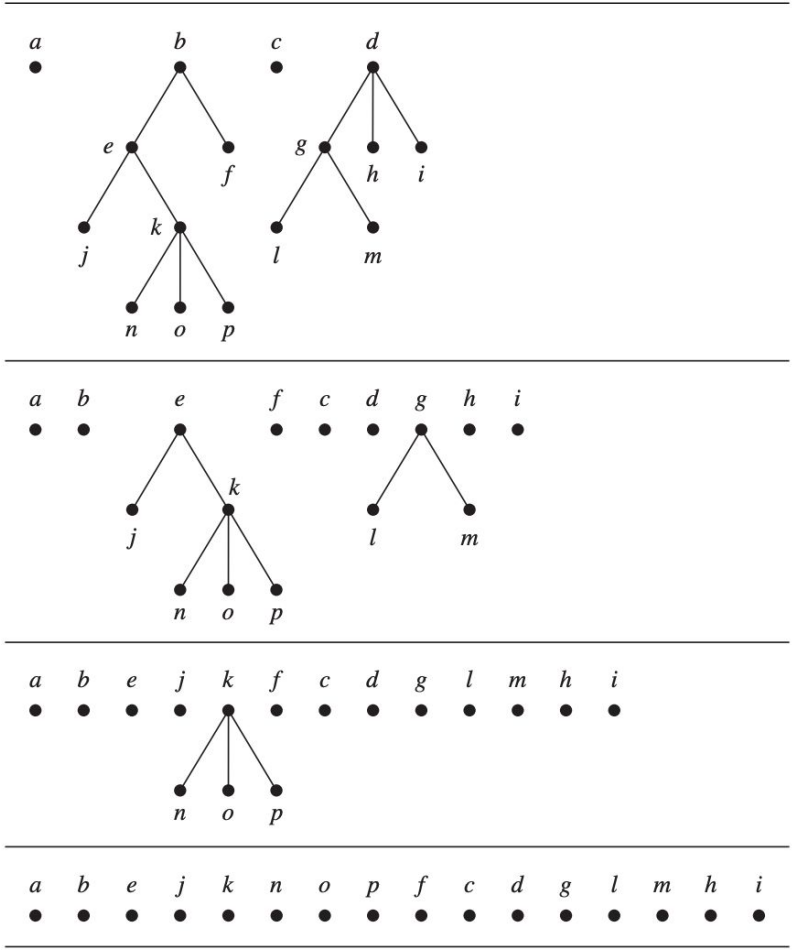






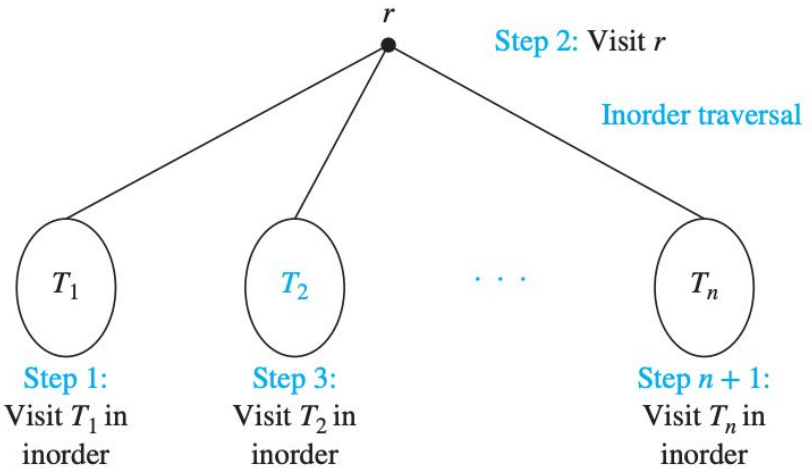
Preorder Traversal (15)





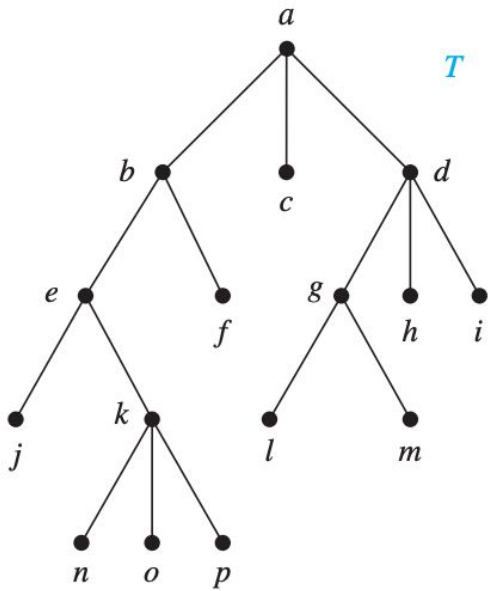
Inorder Traversal

- <u>Definição:</u> Seja T uma árvore enraizada ordenada com raiz r:
  - Se T contém apenas r: inorder(T) = r
  - Caso contrário:
    - Sejam T_1, T_2, ..., T_n as subárvores de r (da esquerda para a direita)
- Algoritmo
  - Percorra T_1 em inorder
  - Visite a raiz r
  - Percorra T_2, T_3, ..., T_n em inorder

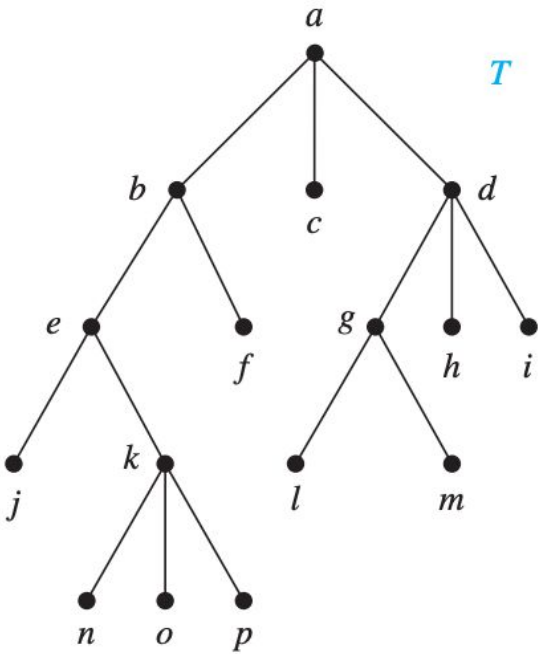


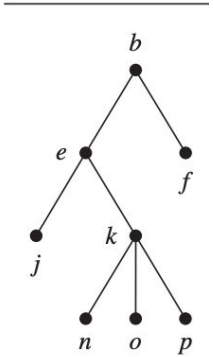
Inorder Traversal (1)

- Problema: Em qual ordem o algoritmo de preorder traversal visita os vértices da árvore T ?
  - Regra do inorder:
    - Visitar subárvore mais à esquerda
    - Visitar raiz
    - Visitar demais subárvores (da esquerda para direita)

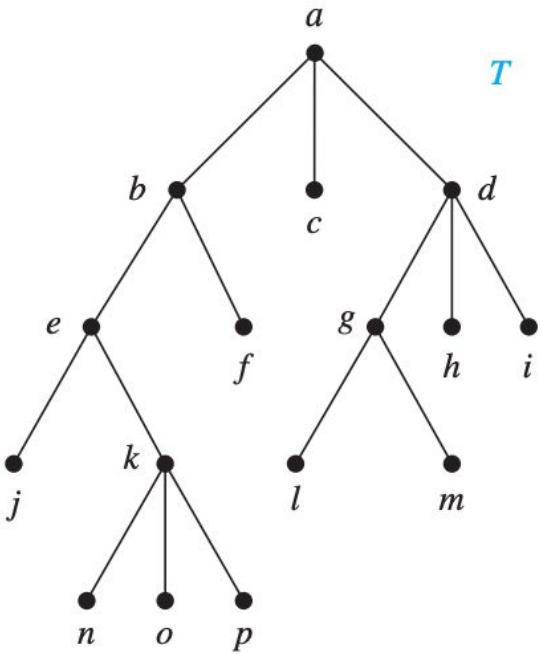


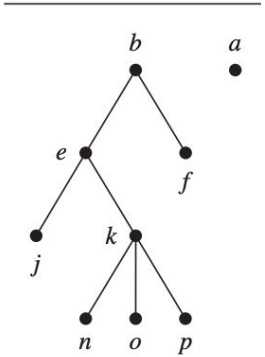
Inorder Traversal (2)



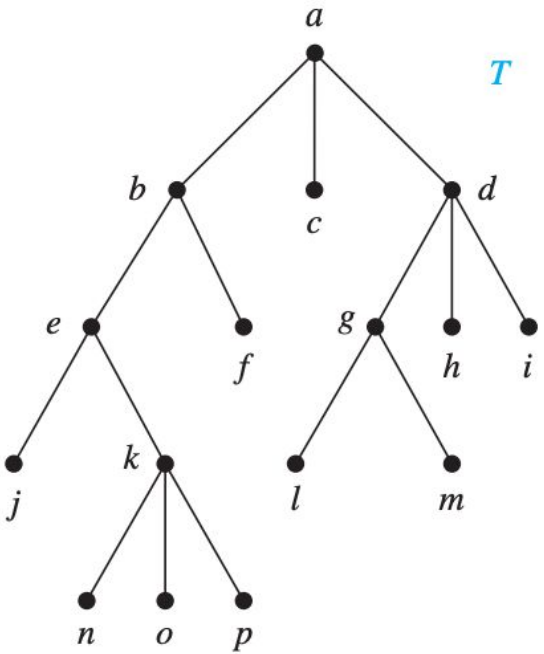


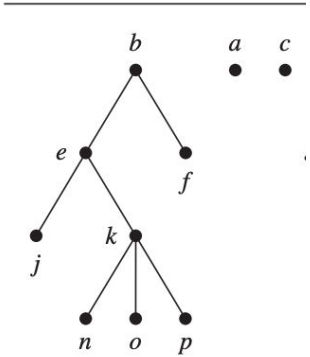
Inorder Traversal (3)



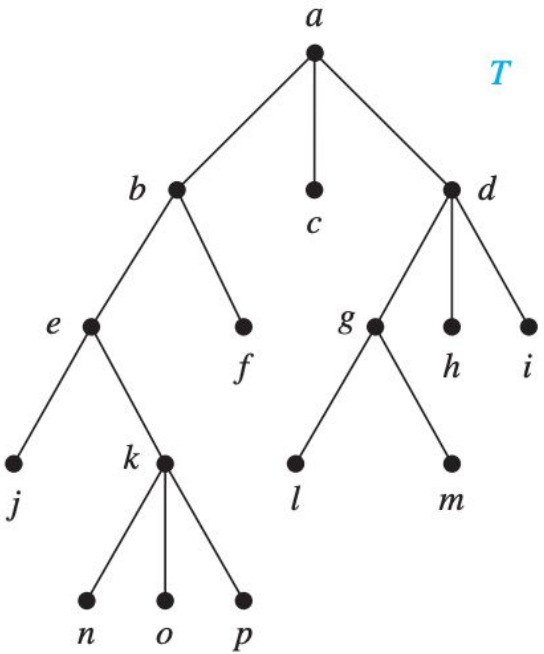


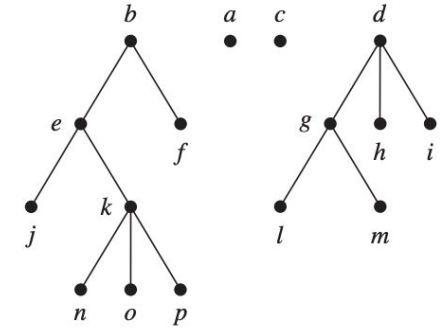
Inorder Traversal (4)



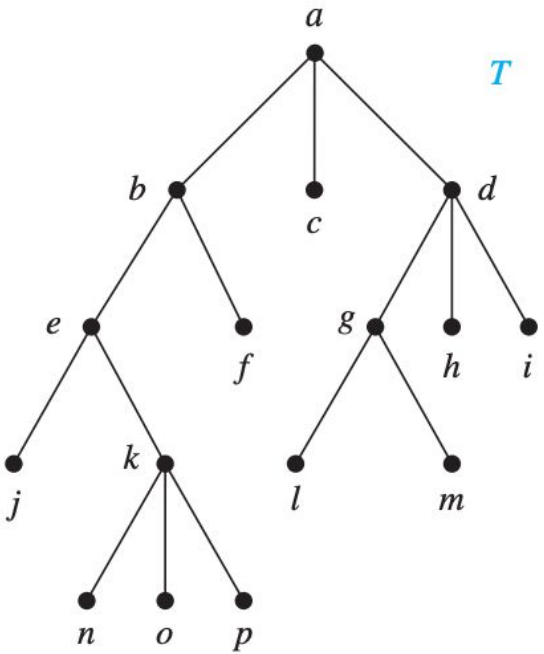


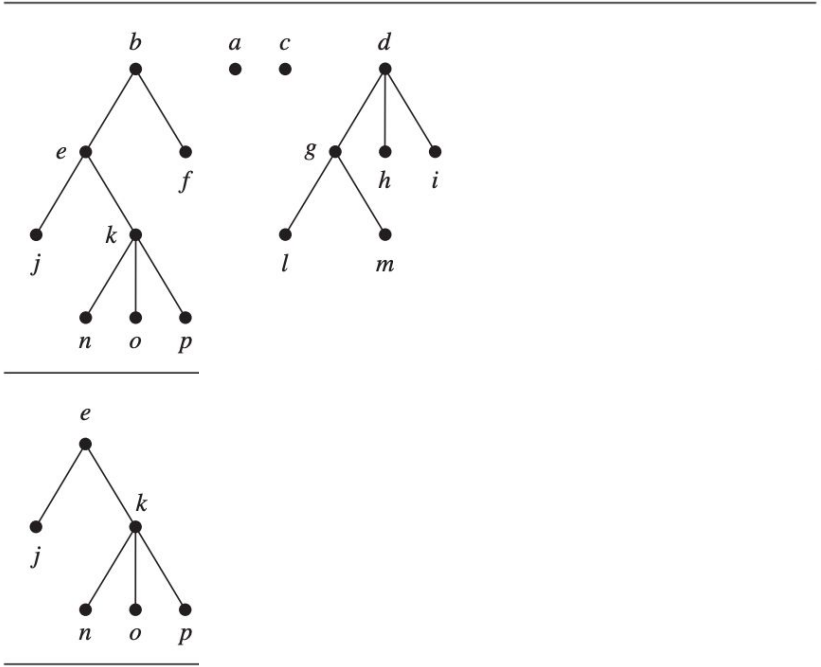
Inorder Traversal (5)



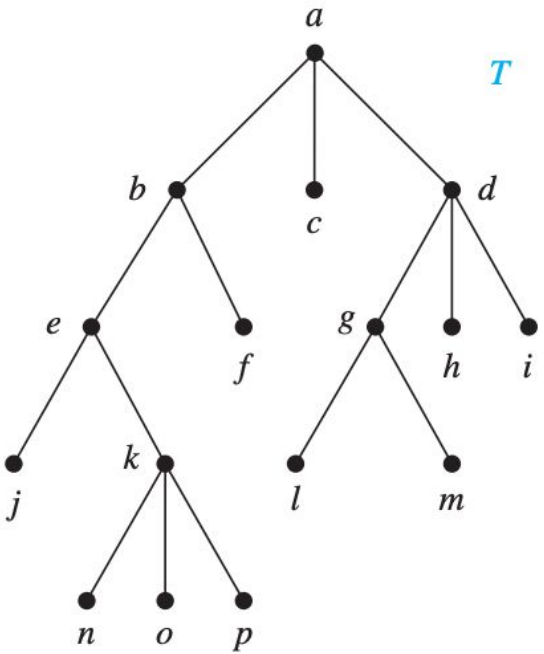


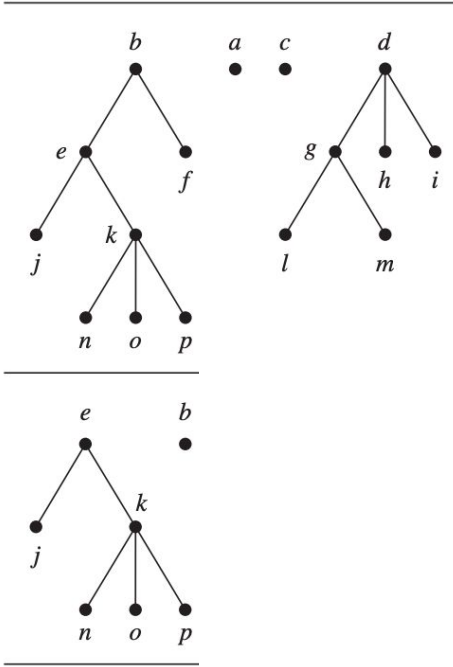
Inorder Traversal (6)



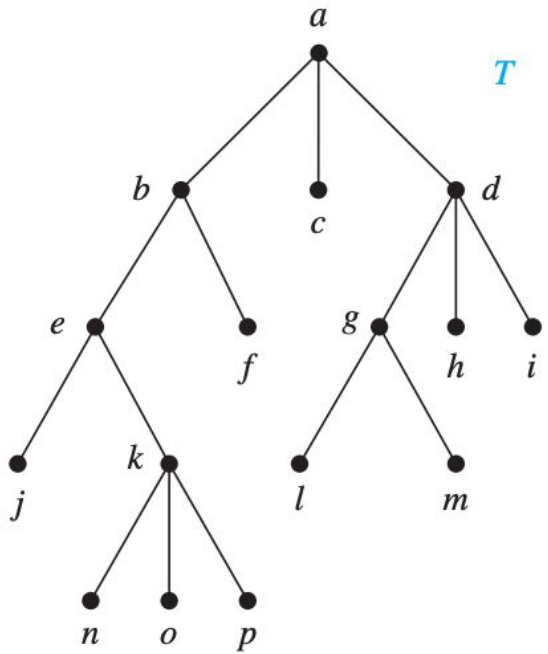


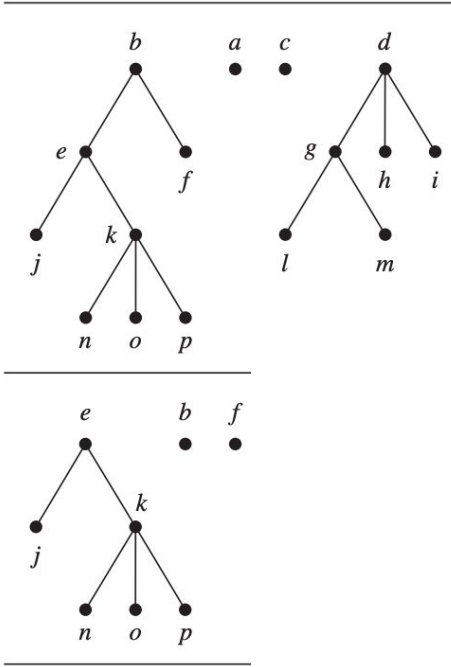
Inorder Traversal (7)



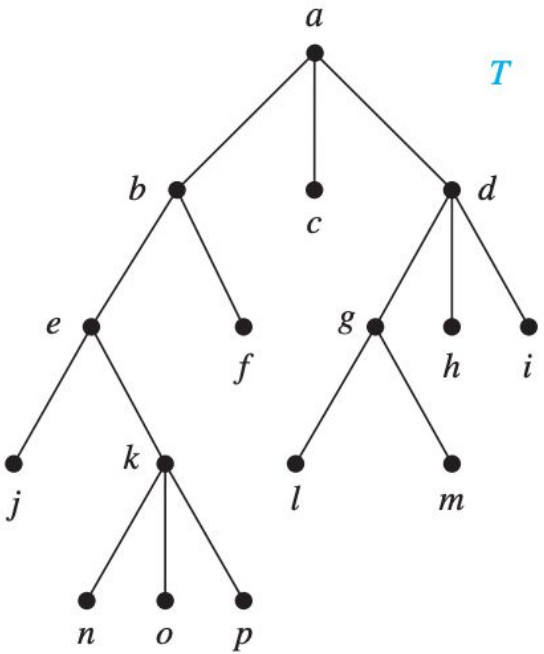


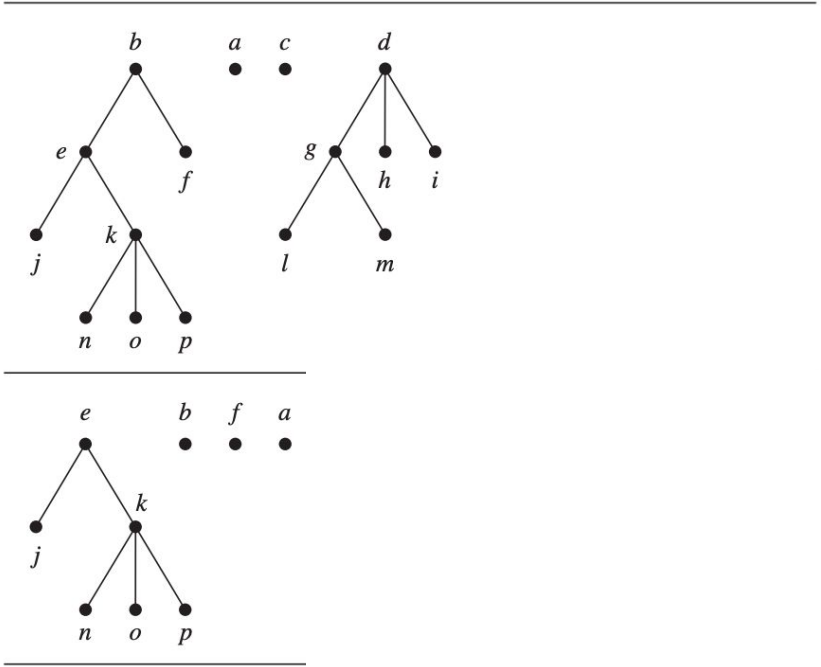
Inorder Traversal (8)



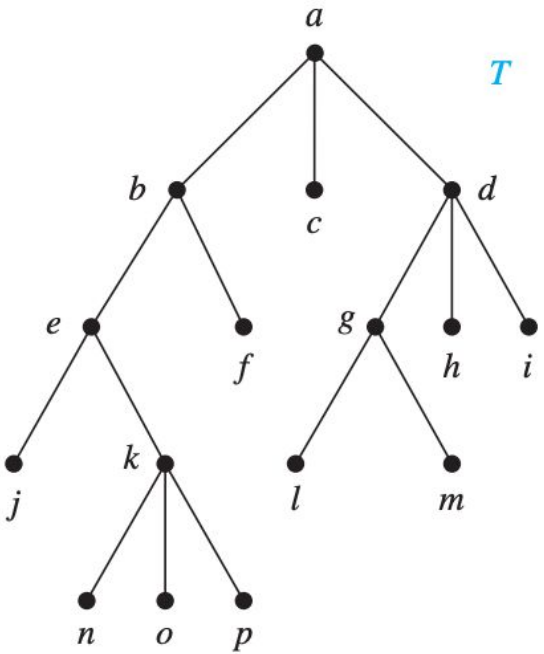


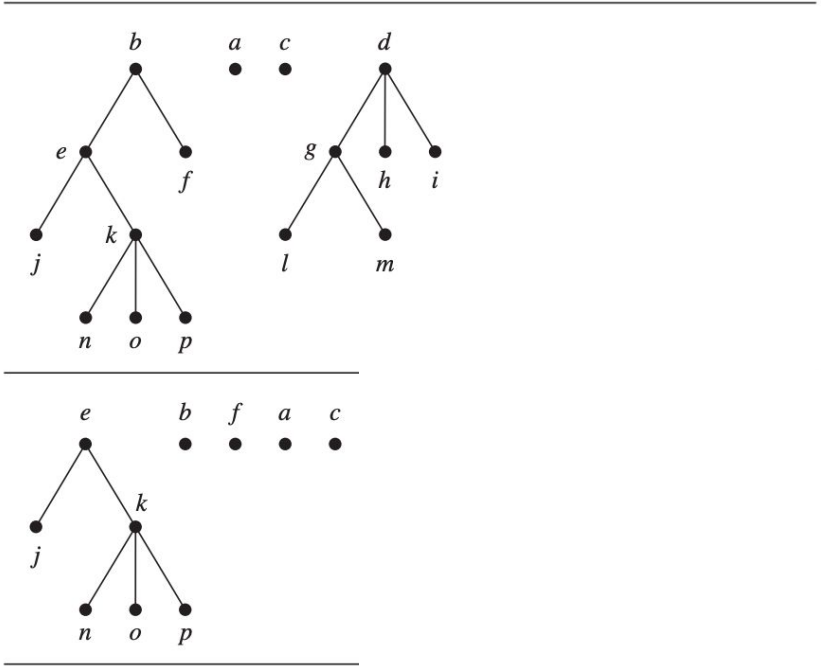
Inorder Traversal (9)



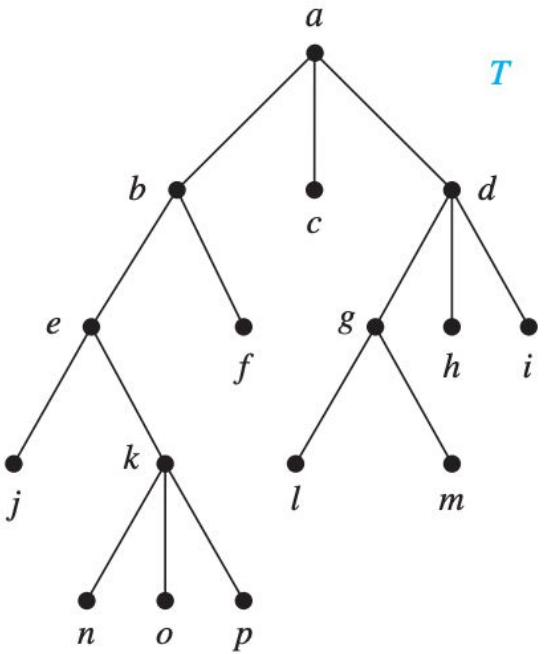


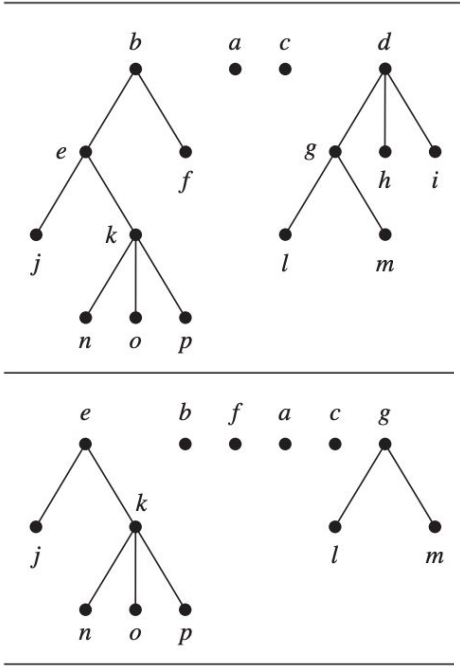
Inorder Traversal (10)



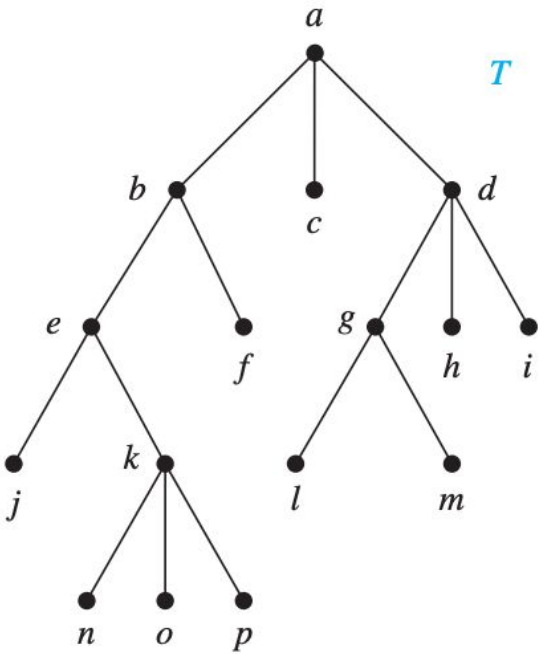


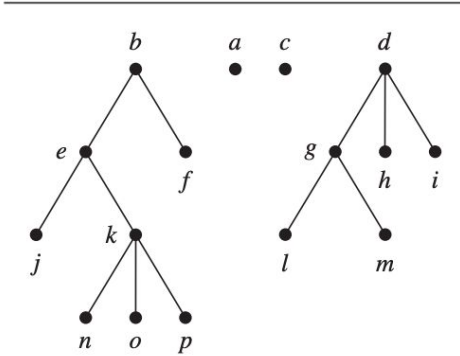
Inorder Traversal (11)

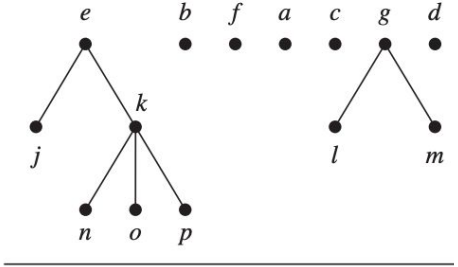




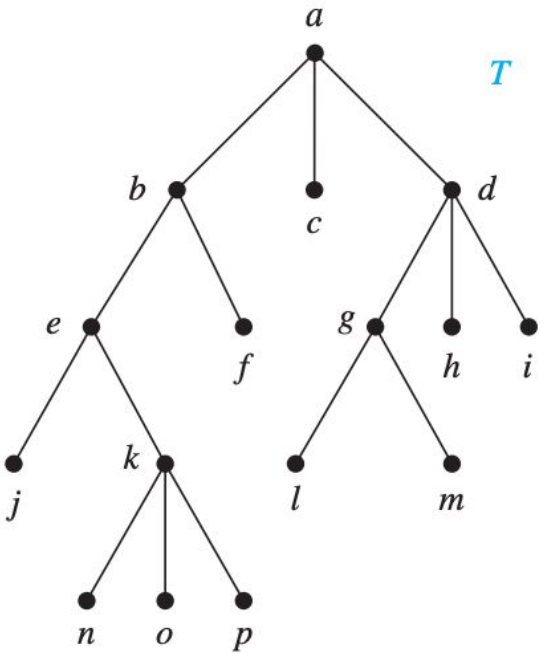
Inorder Traversal (12)

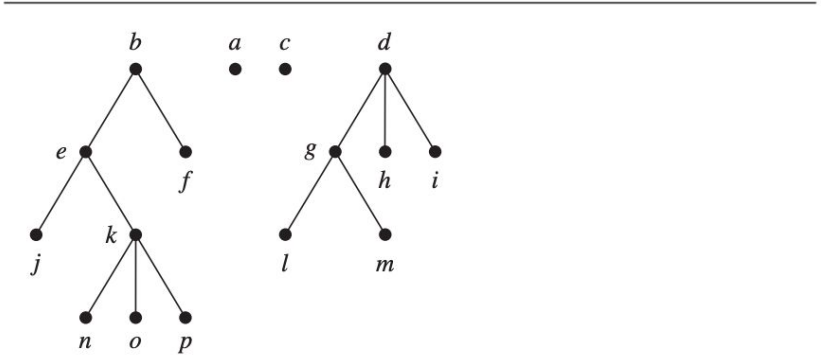


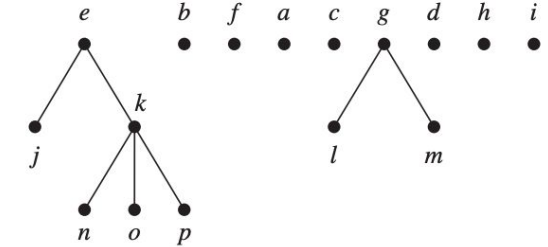




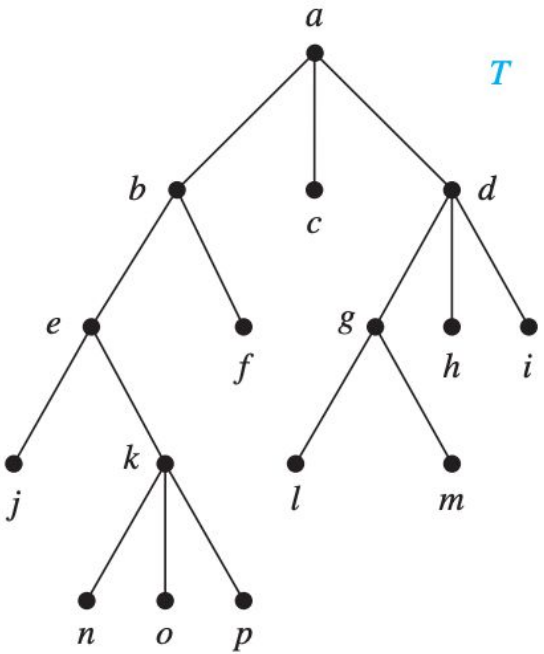
Inorder Traversal (13)

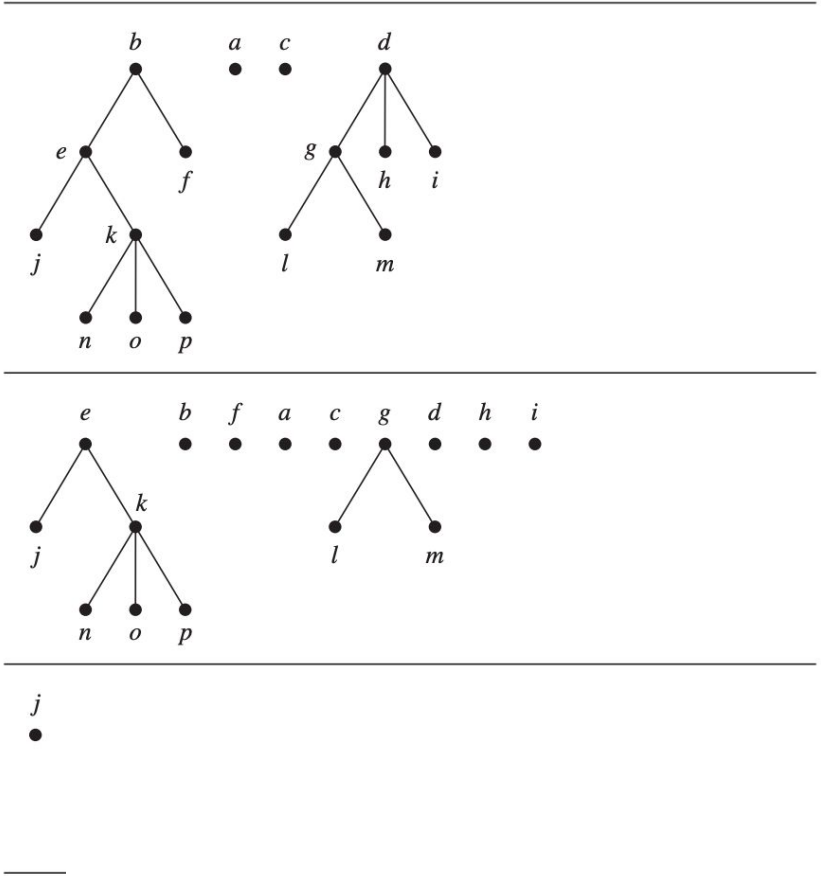




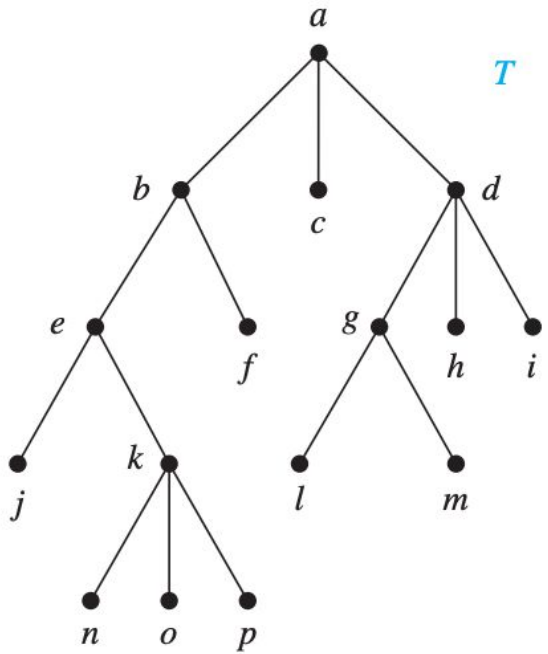


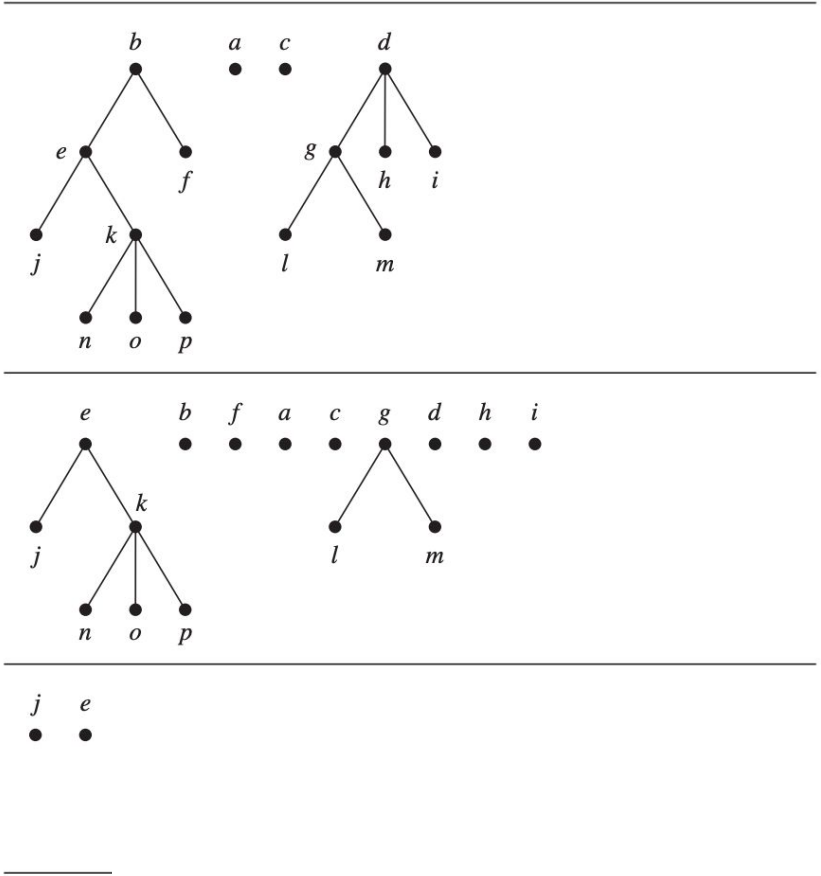
Inorder Traversal (14)



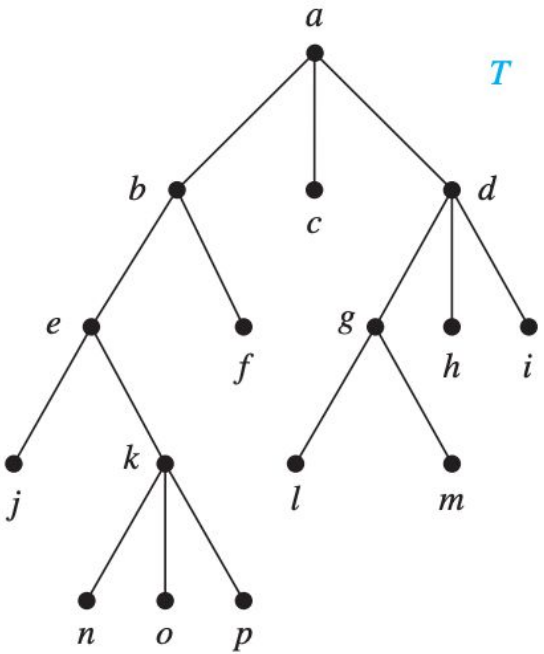


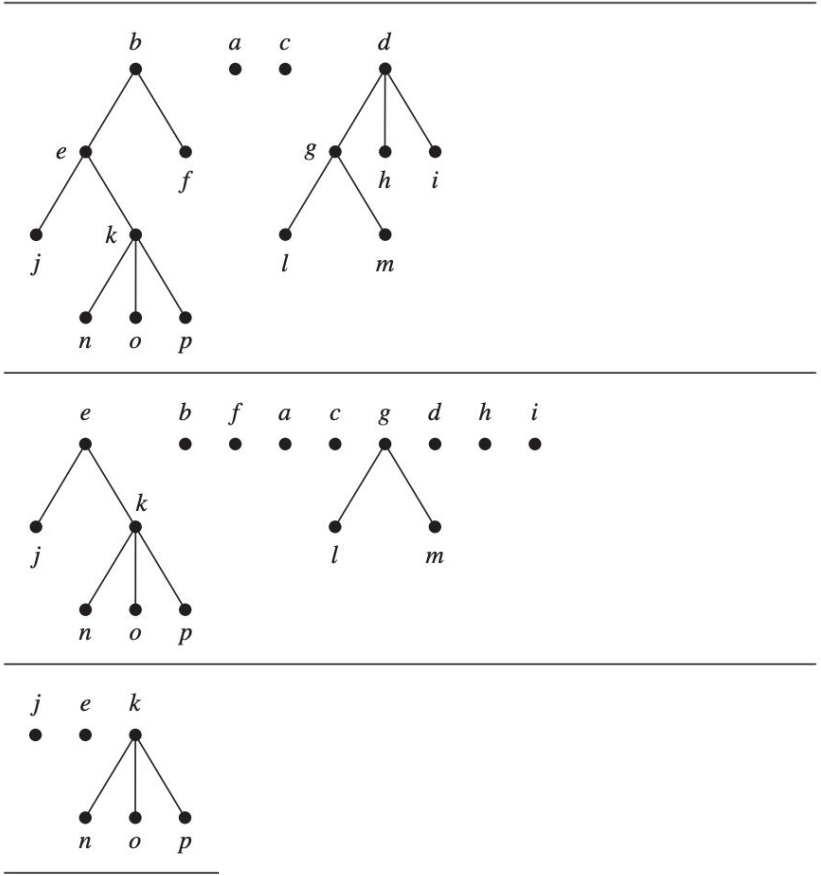
Inorder Traversal (15)



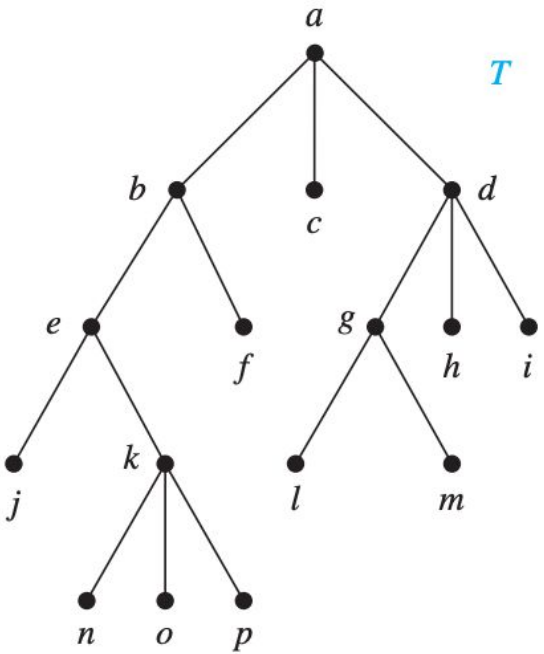


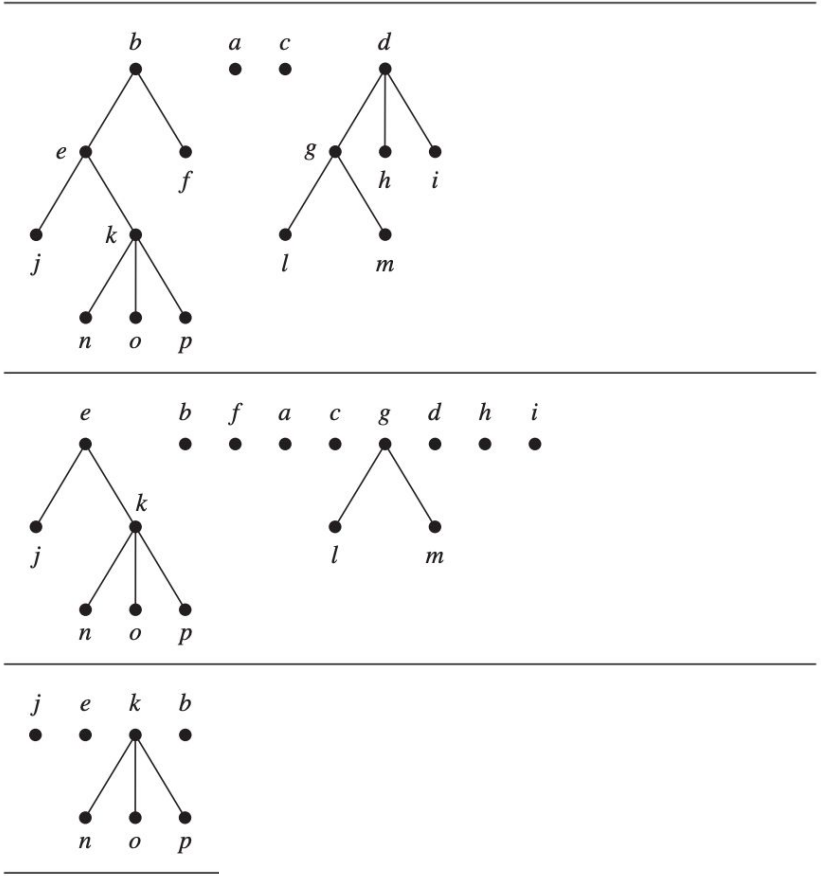
Inorder Traversal (16)



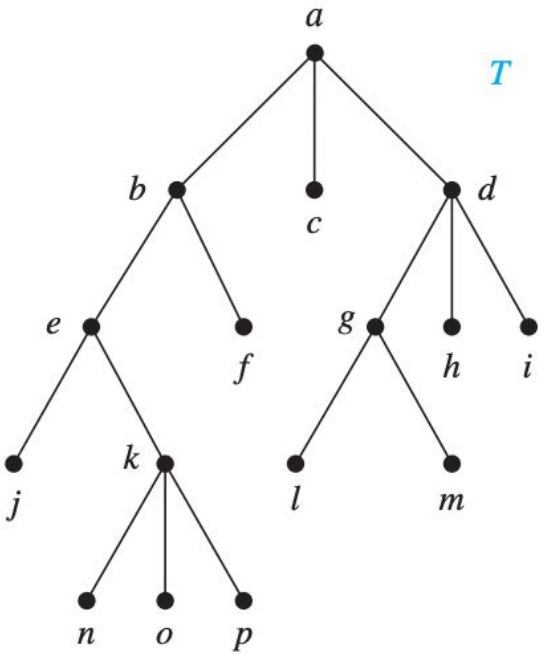


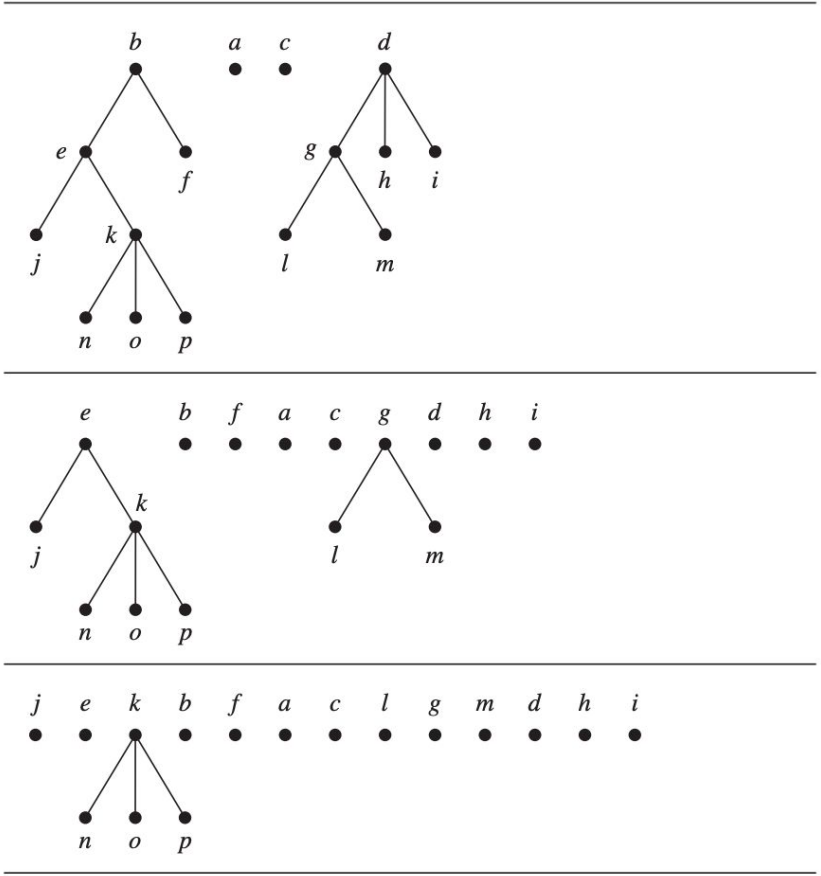
Inorder Traversal (17)



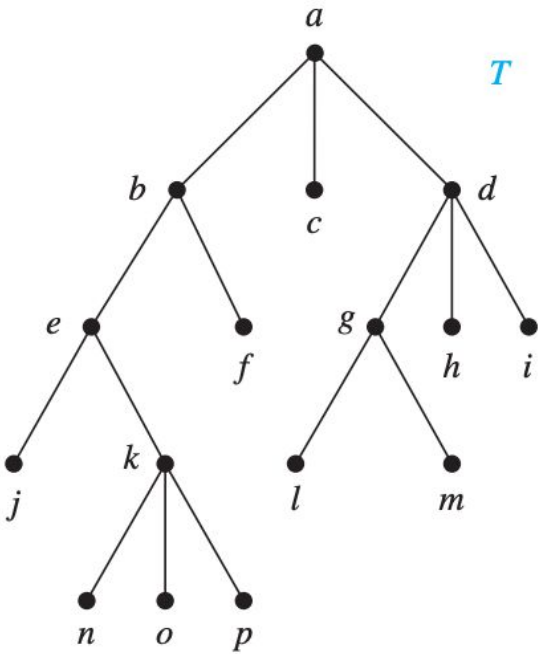


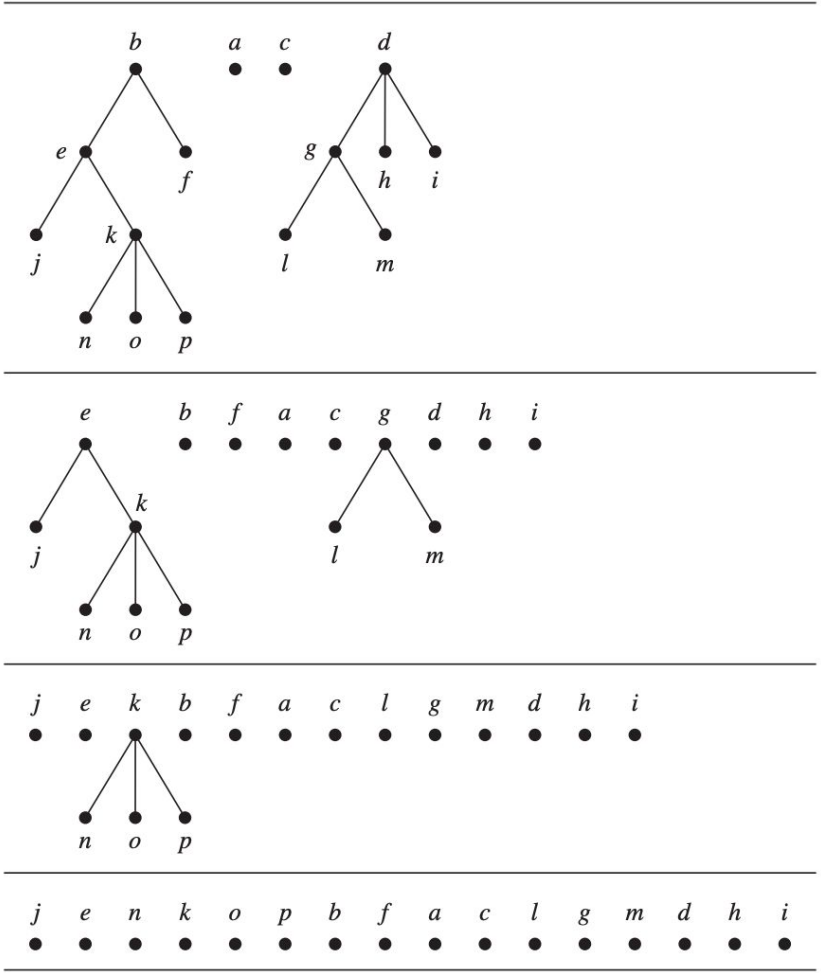
Inorder Traversal (18)





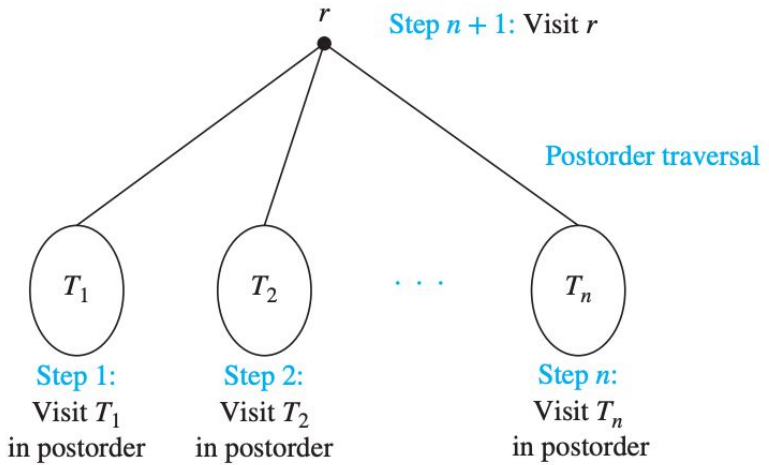
Inorder Traversal (19)





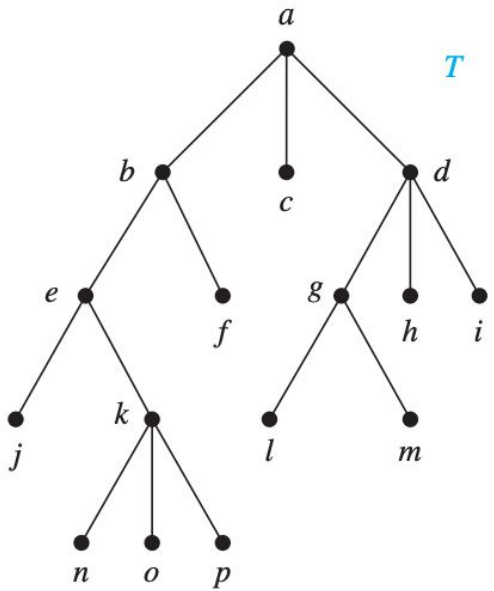
Postorder Traversal

- <u>Definição:</u> Seja T uma árvore enraizada ordenada com raiz r:
  - Se T contém apenas r: postorder(T) = r
  - Caso contrário:
    - Sejam T_1, T_2, ..., T_n as subárvores de r (da esquerda para a direita)
- Algoritmo
  - Percorra T_1 em postorder
  - Percorra T_2, T_3, ..., T_n em postorder
  - Visite a raiz r

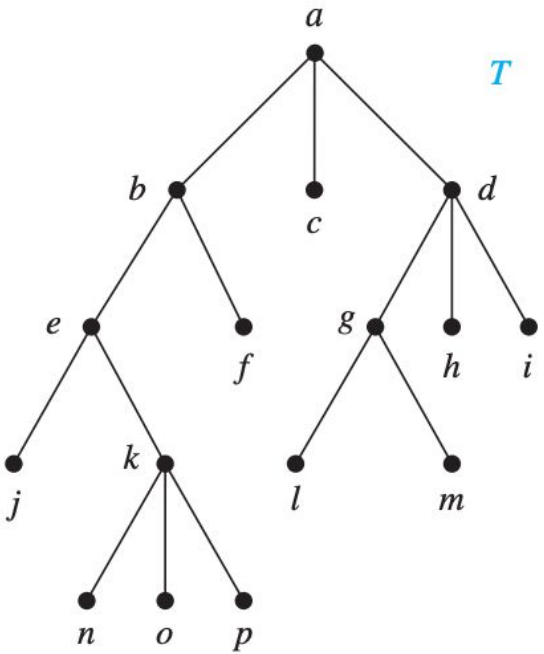


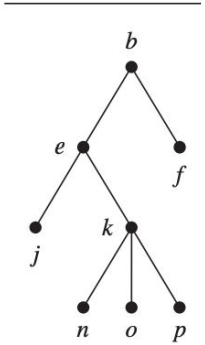
Postorder Traversal (1)

- Problema: Em qual ordem o algoritmo de preorder traversal visita os vértices da árvore T ?
  - Regra do inorder:
    - Visitar subárvore mais à esquerda
    - Visitar raiz
    - Visitar demais subárvores (da esquerda para direita)

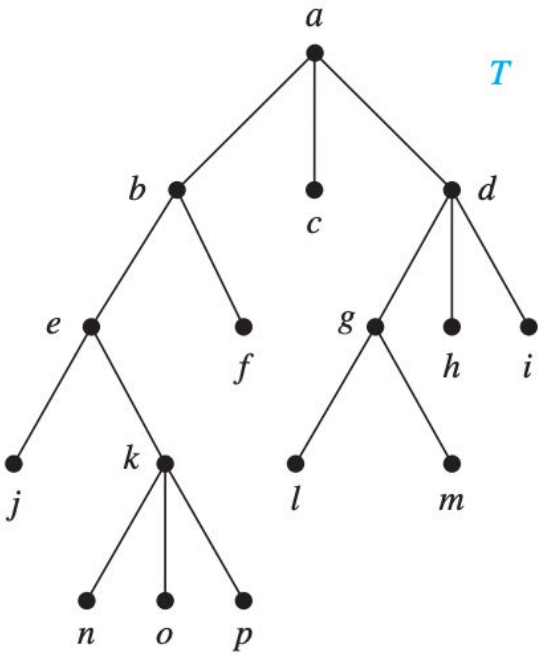


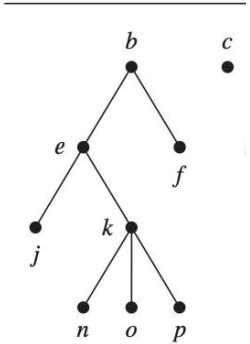
Postorder Traversal (2)



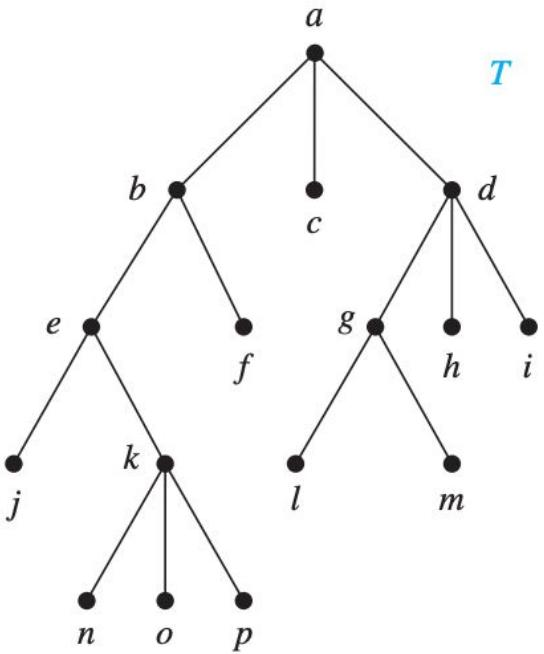


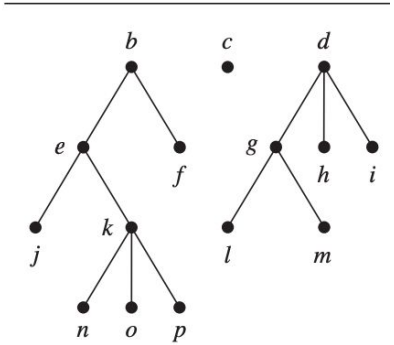
Postorder Traversal (3)



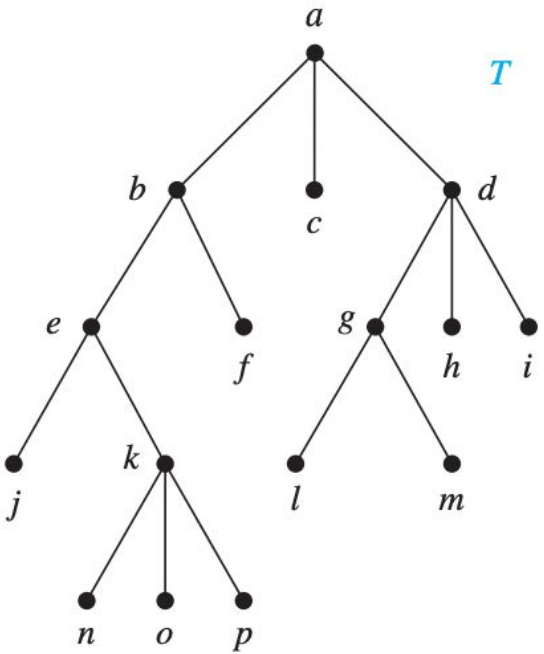


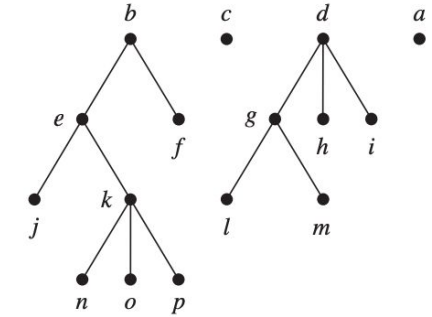
Postorder Traversal (4)



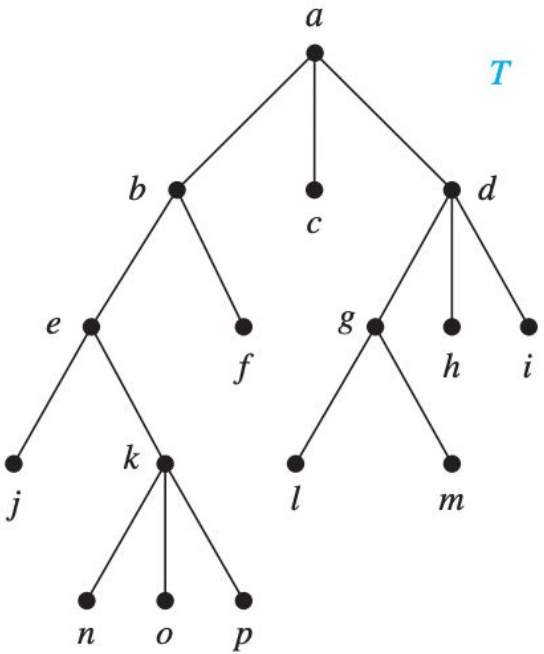


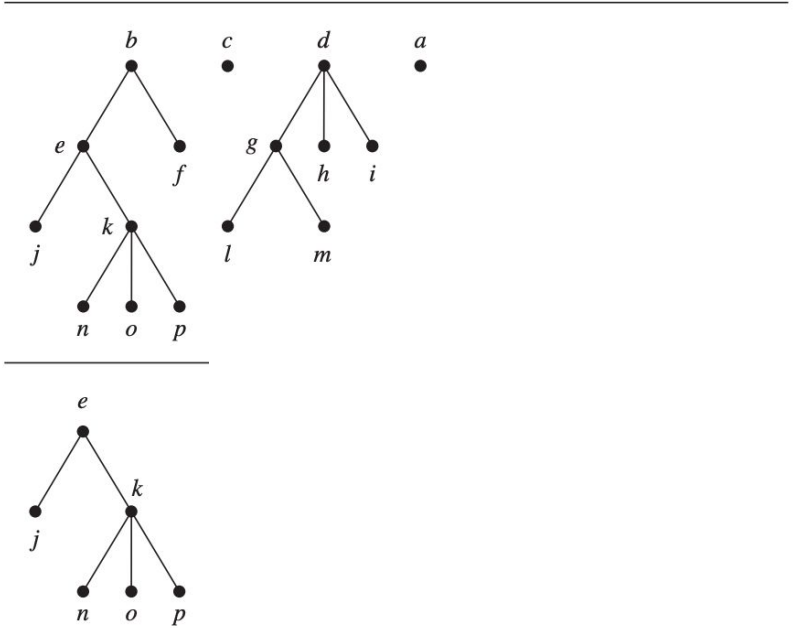
Postorder Traversal (5)



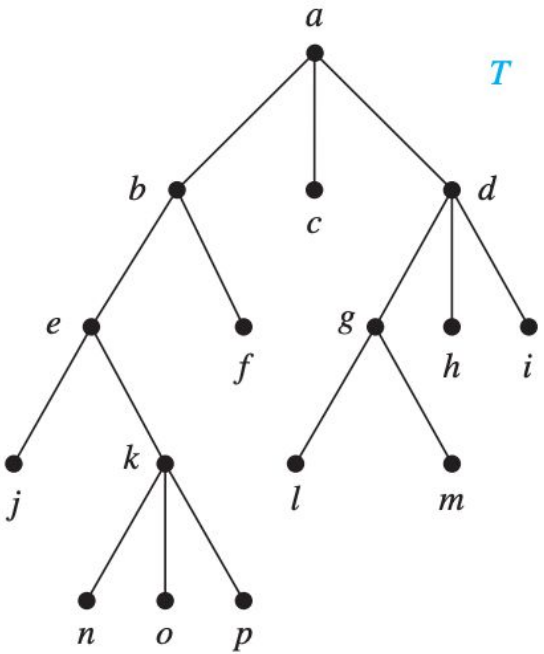


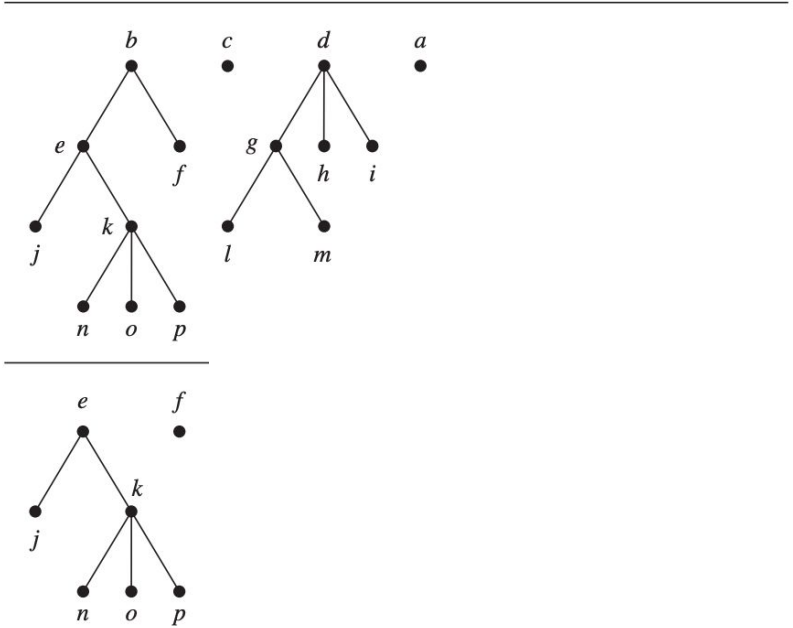
Postorder Traversal (6)



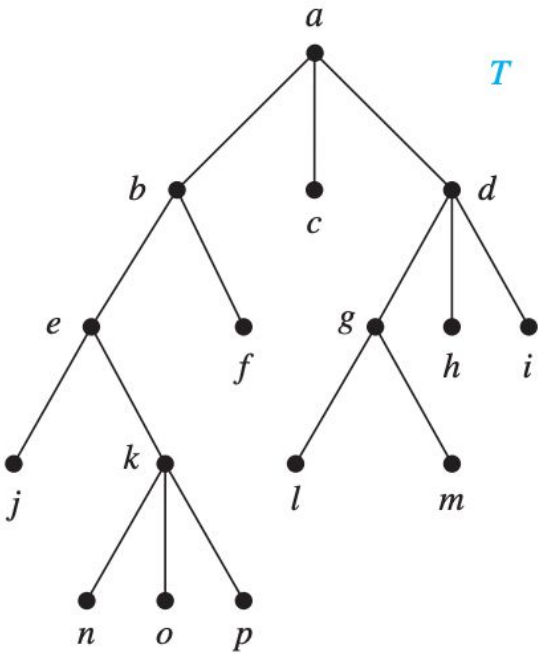


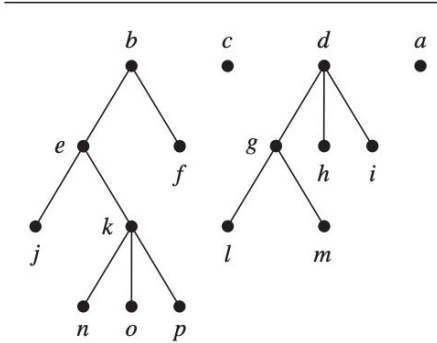
Postorder Traversal (7)

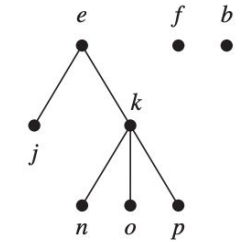




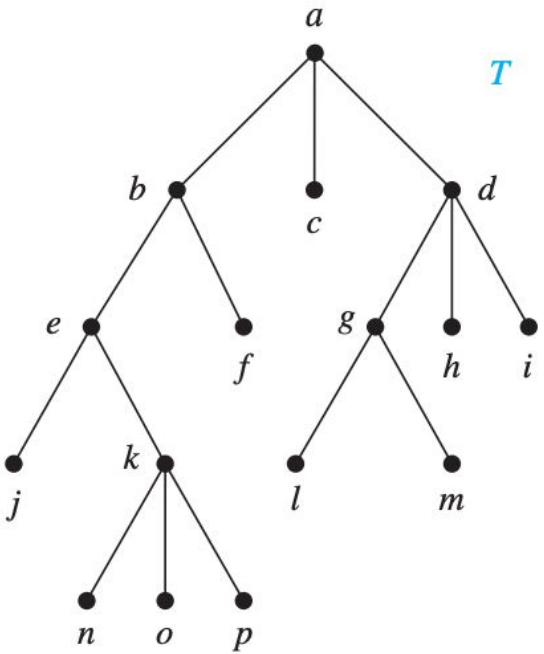
Postorder Traversal (8)

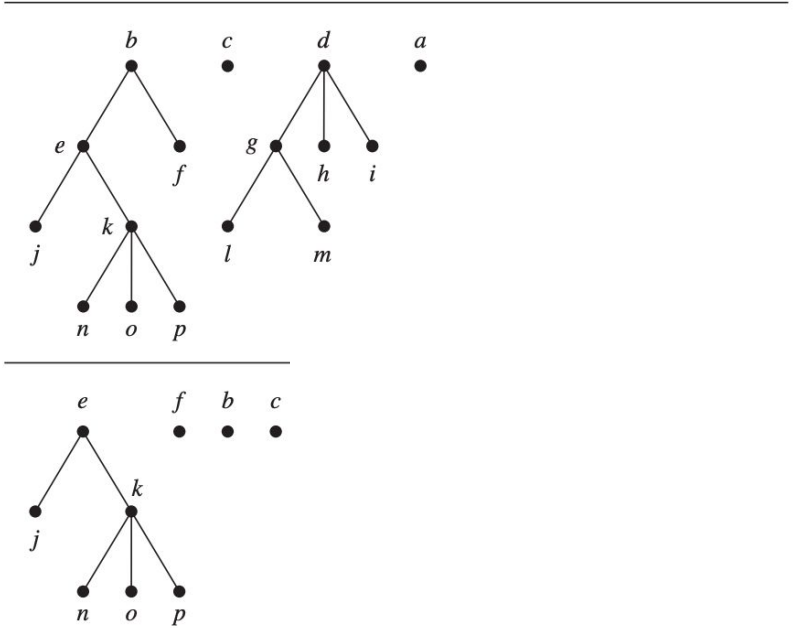




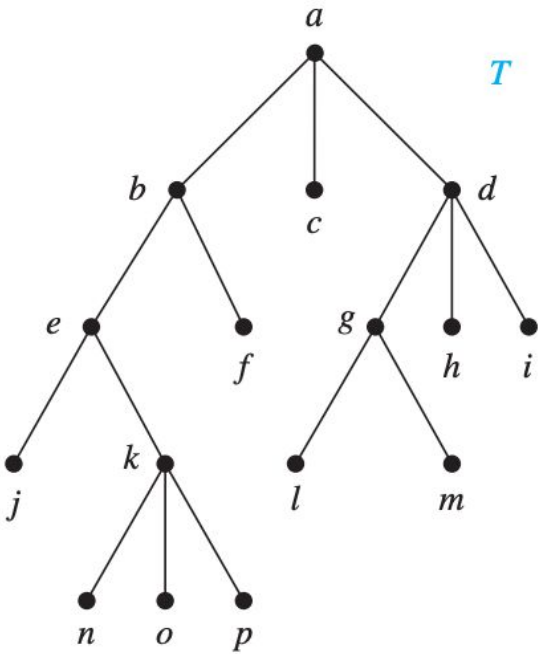


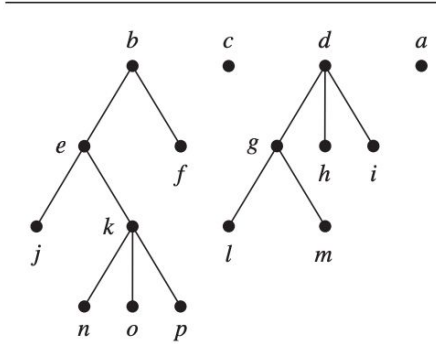
Postorder Traversal (9)

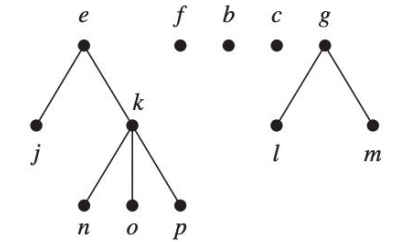




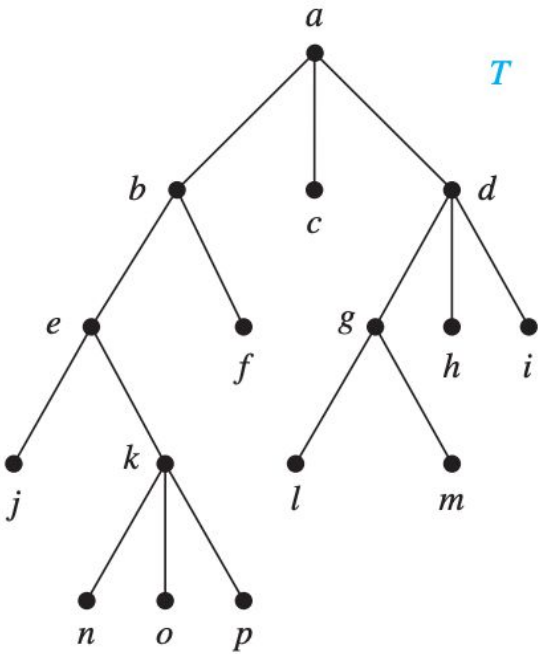
Postorder Traversal (10)

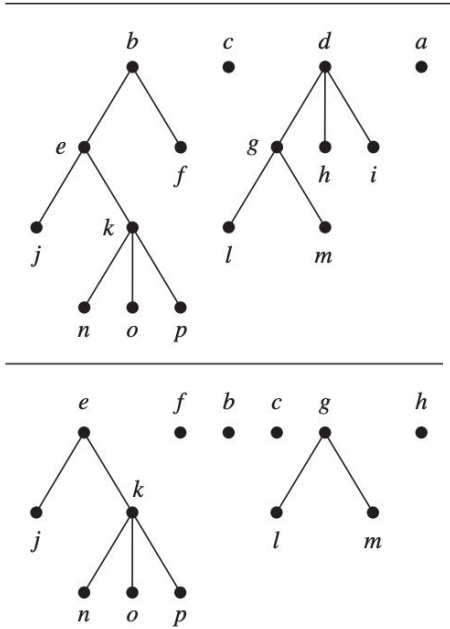




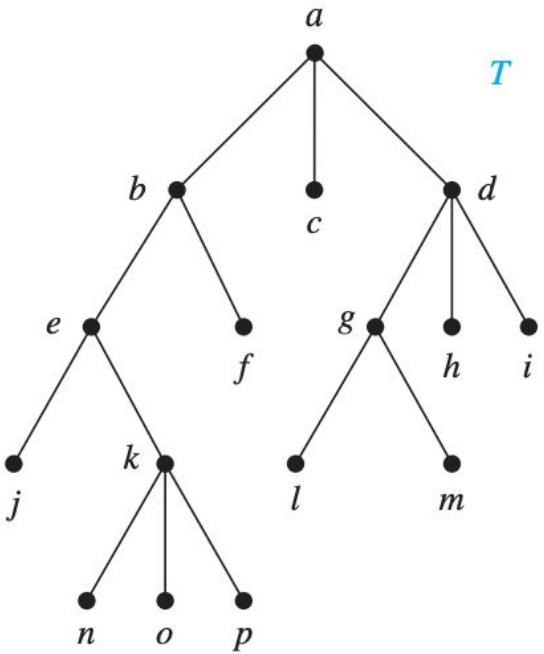


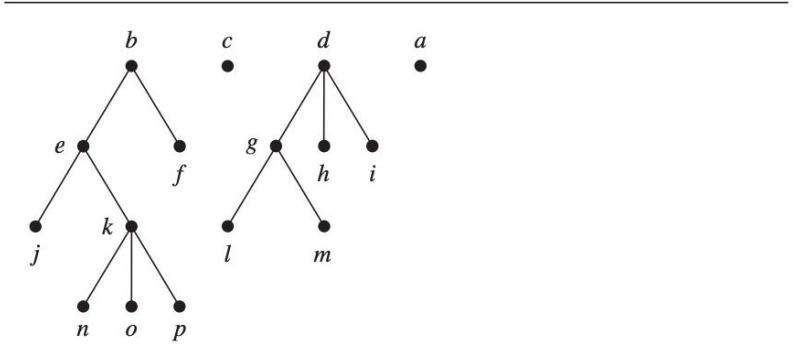
Postorder Traversal (11)

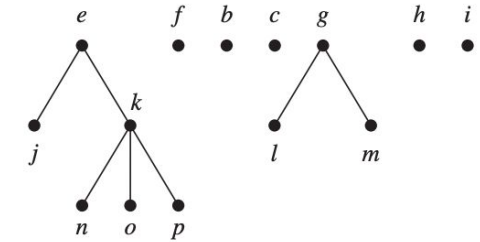




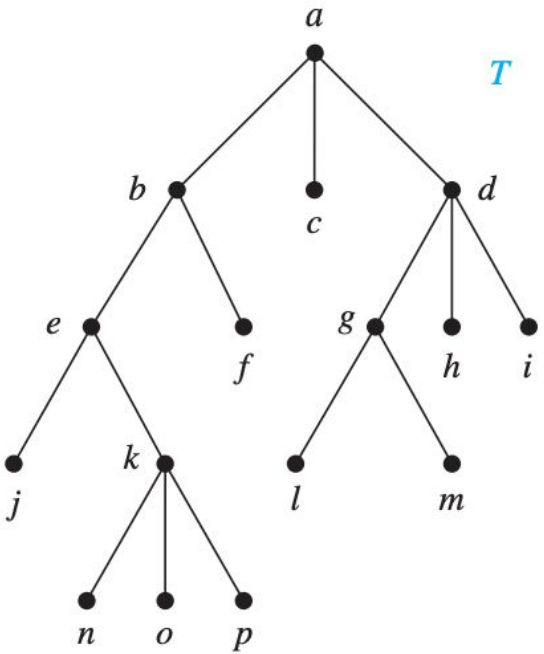
Postorder Traversal (12)

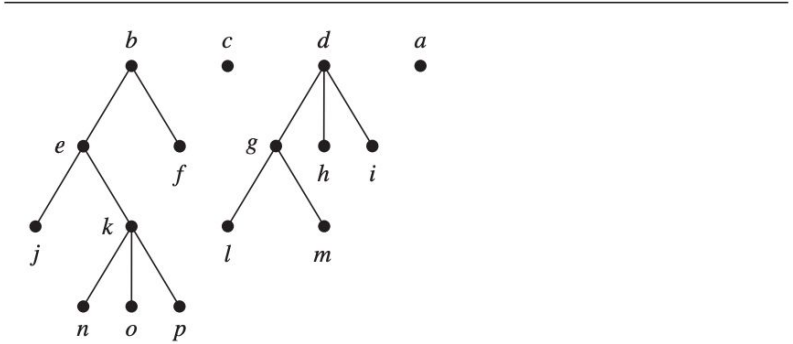


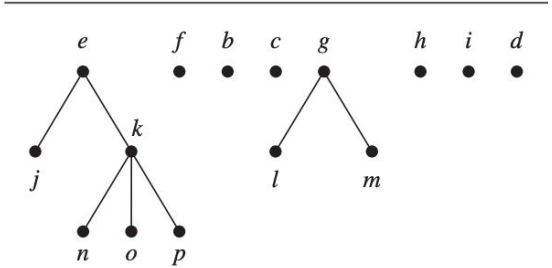




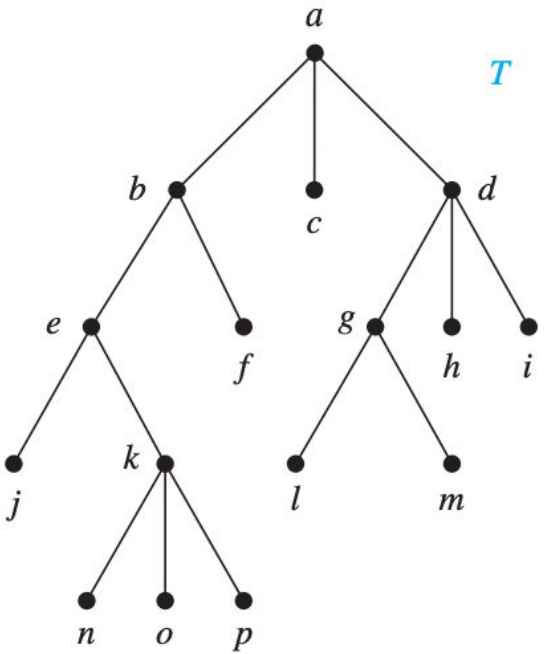
Postorder Traversal (13)

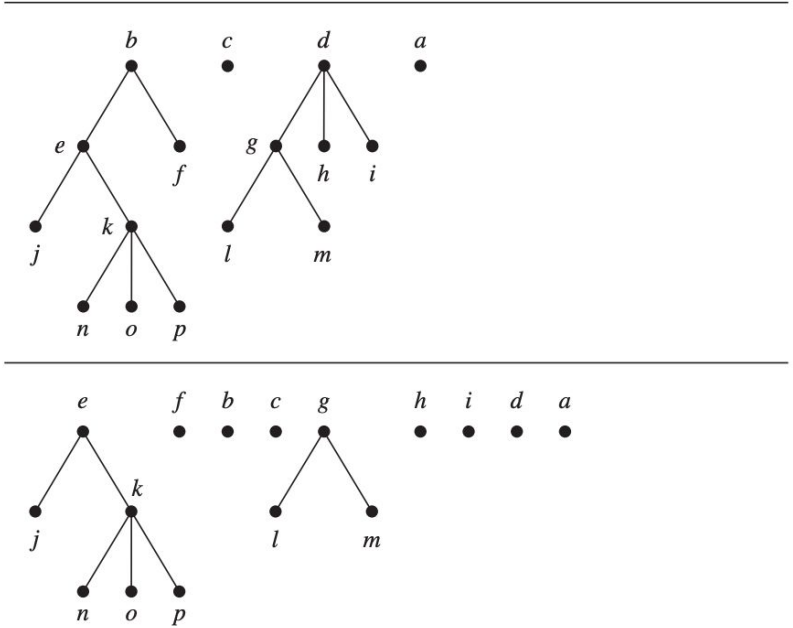




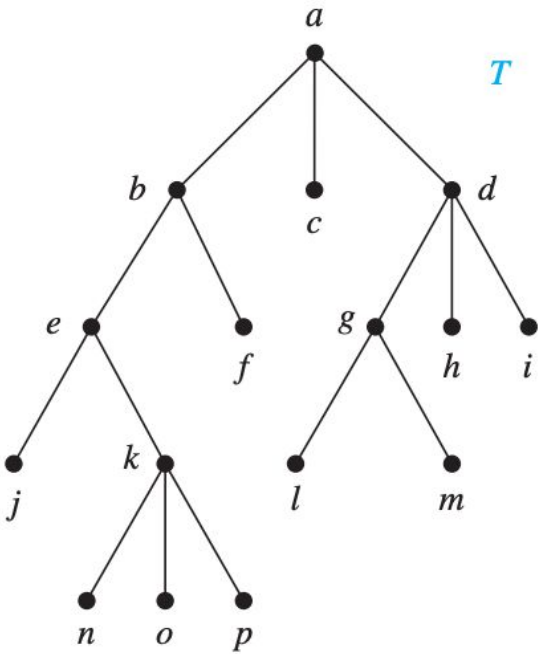


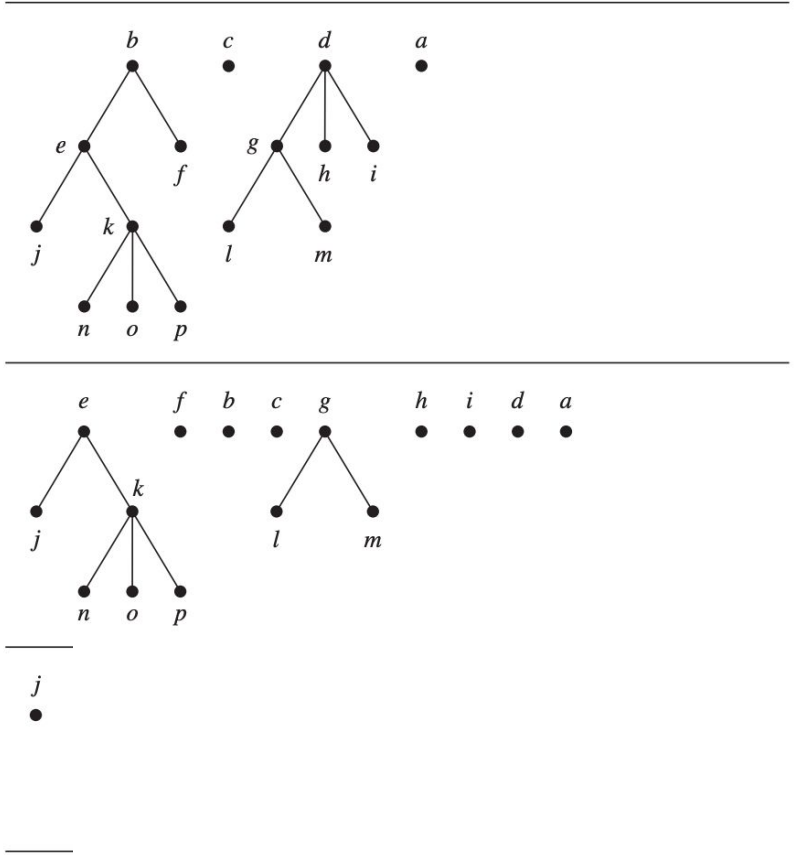
Postorder Traversal (14)



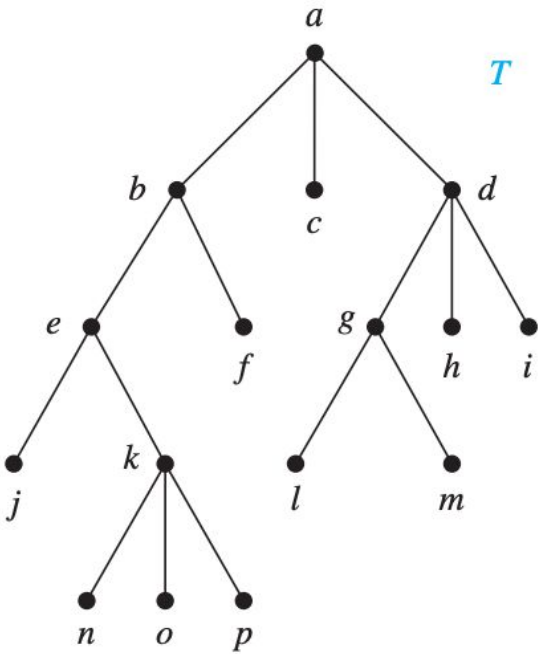


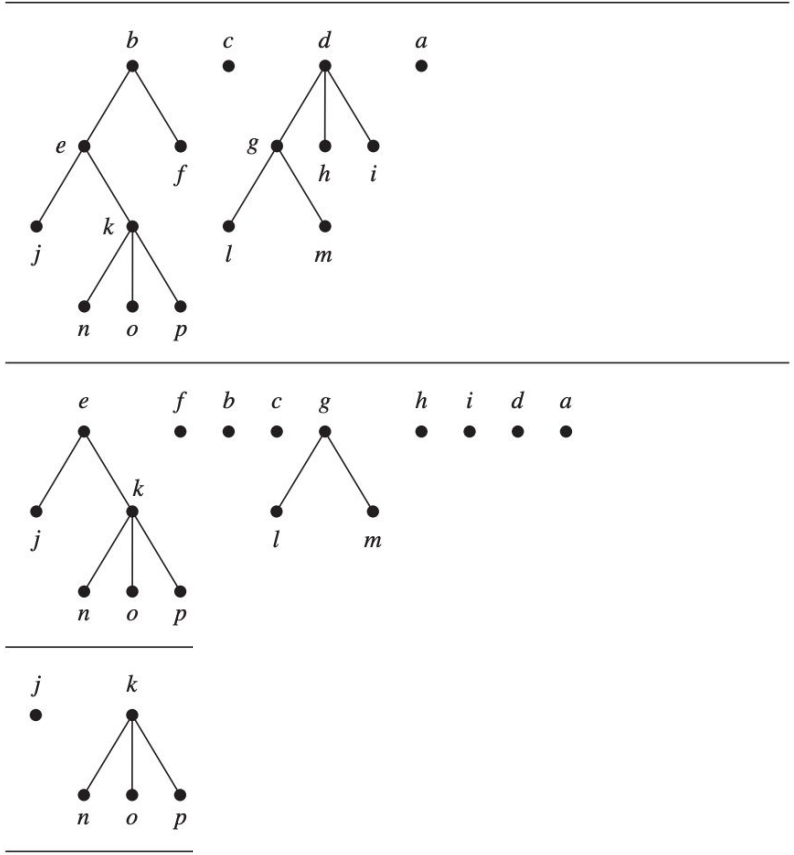
Postorder Traversal (15)



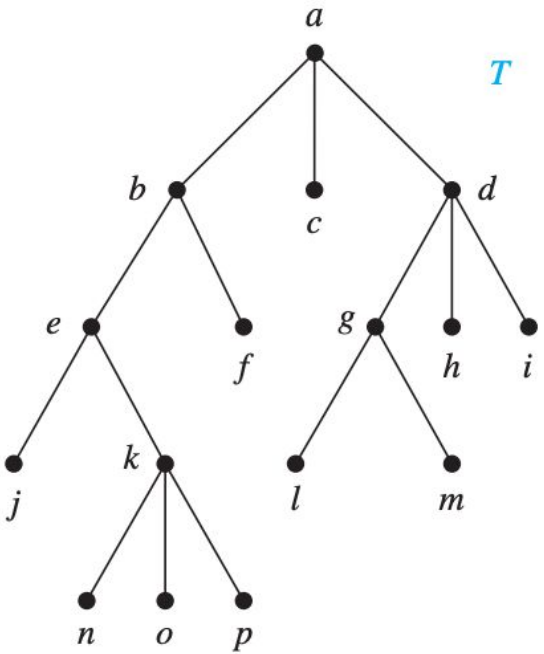


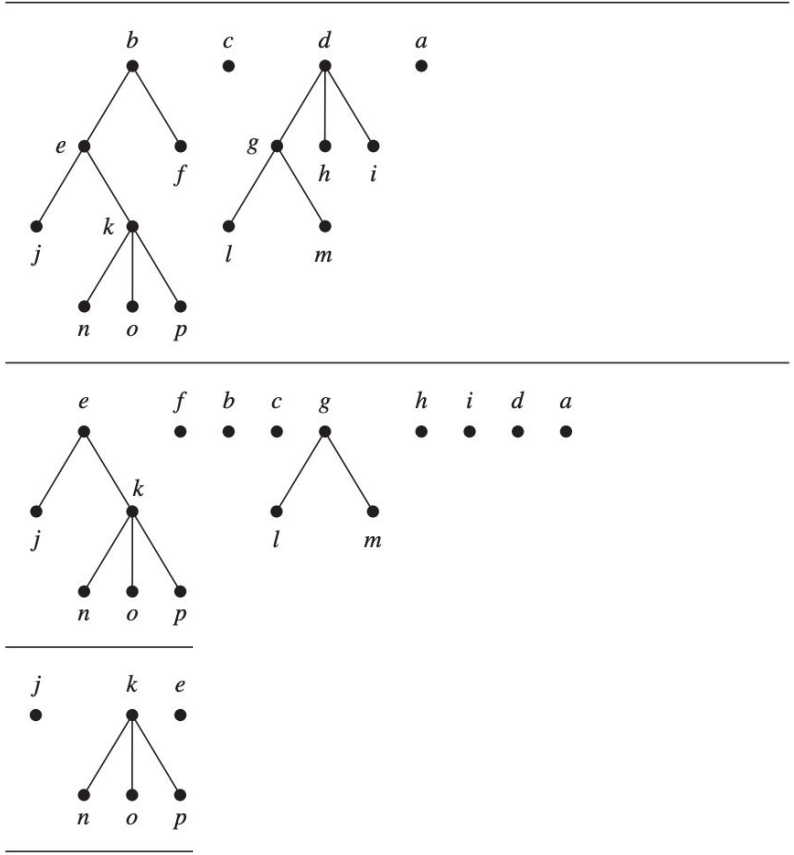
Postorder Traversal (16)



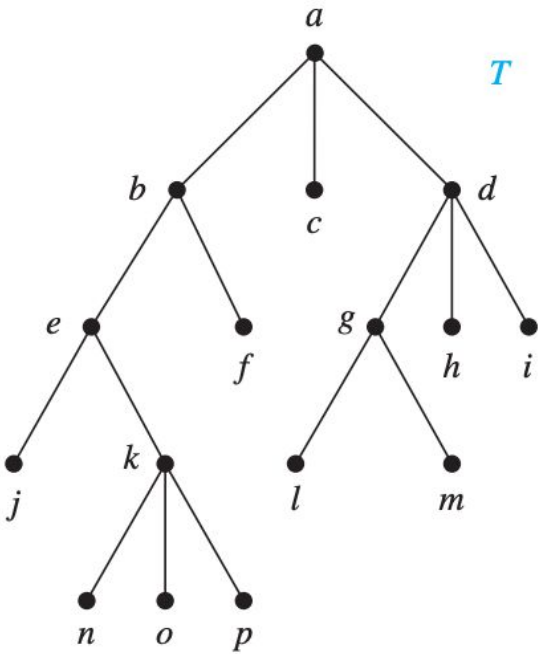


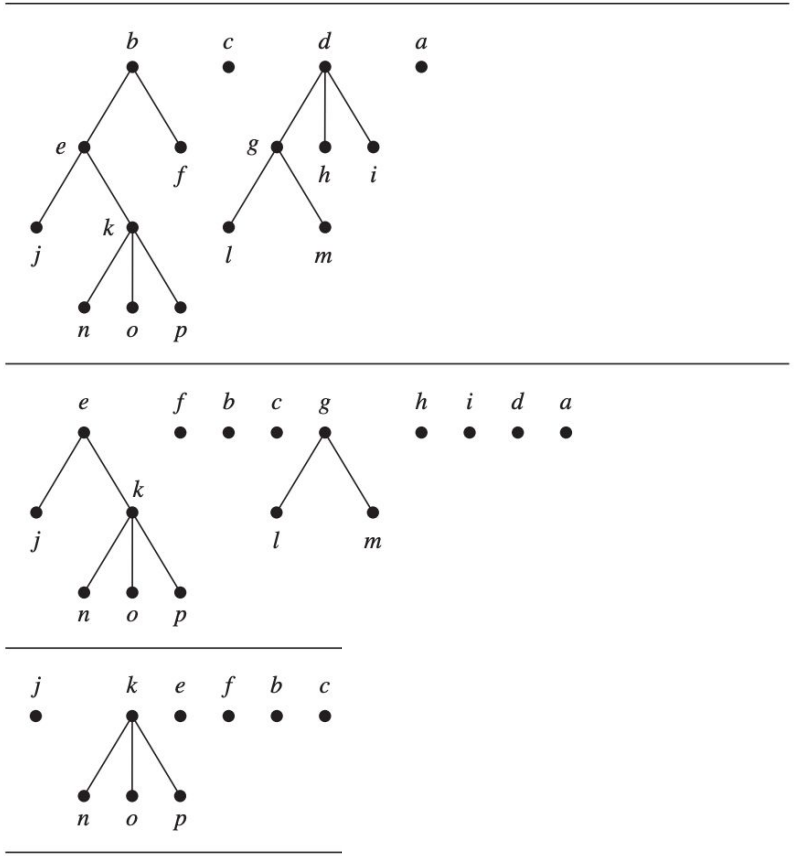
Postorder Traversal (17)



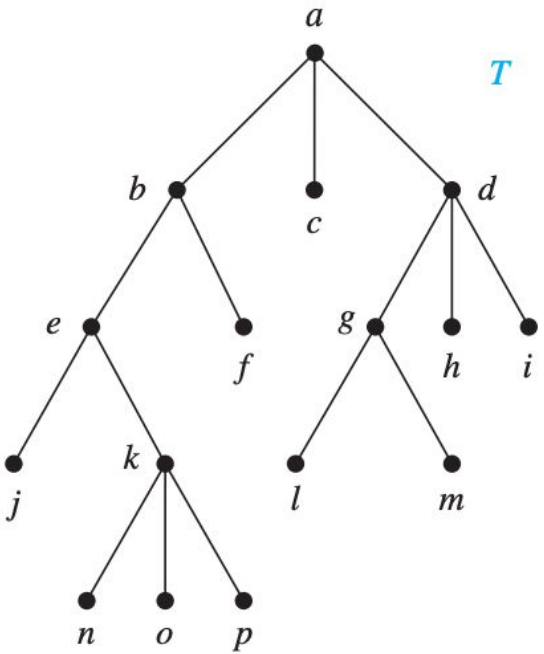


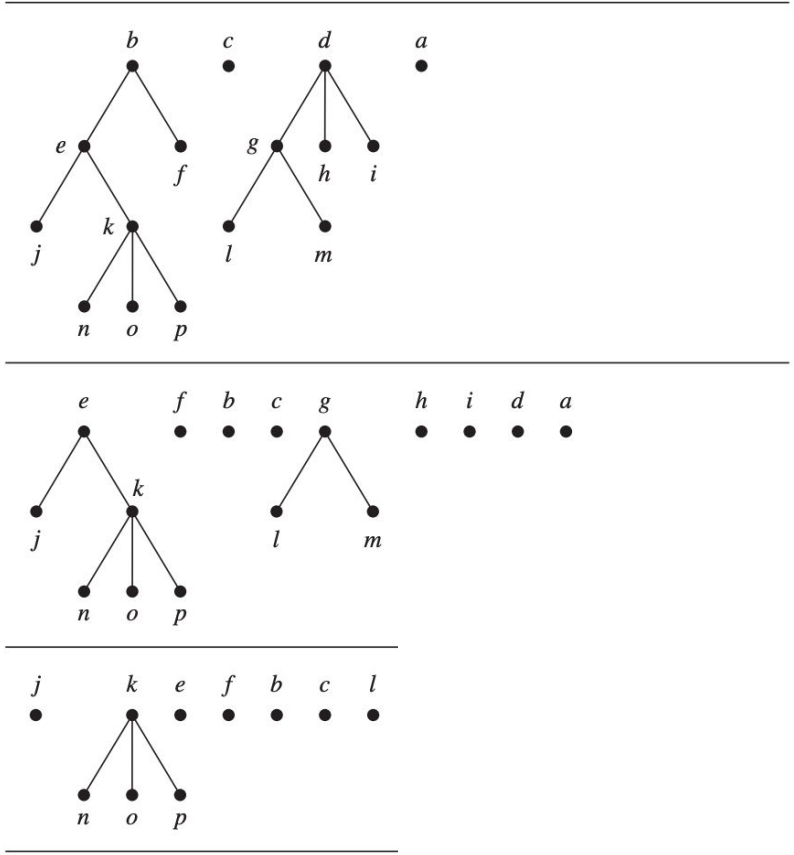
Postorder Traversal (18)



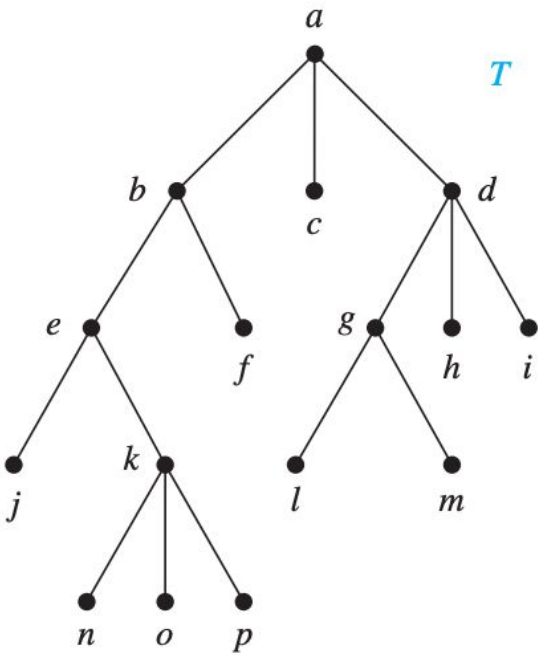


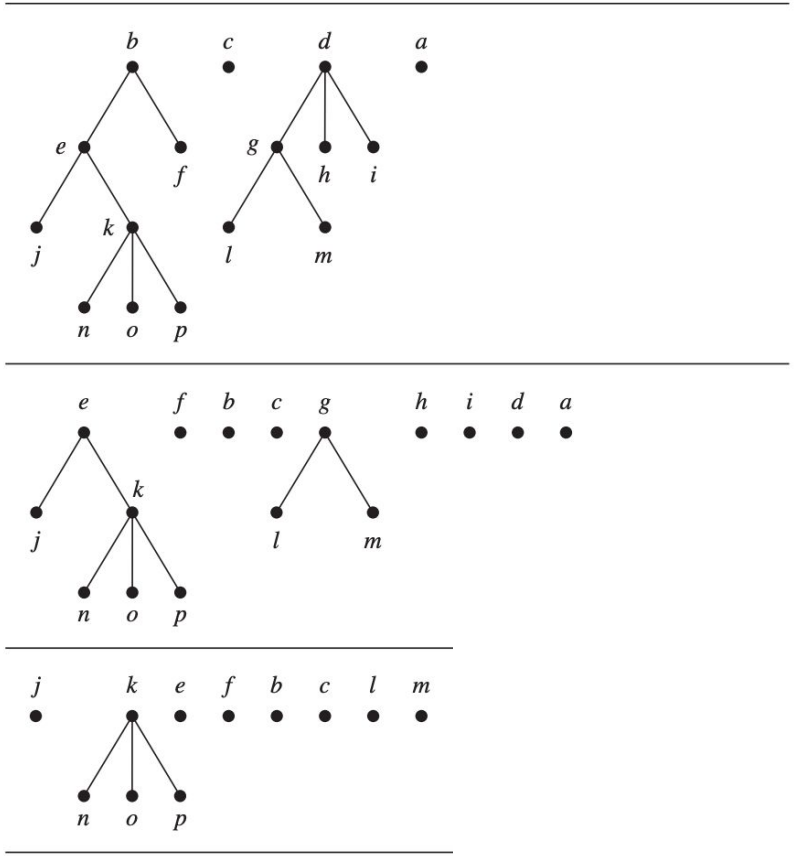
Postorder Traversal (19)



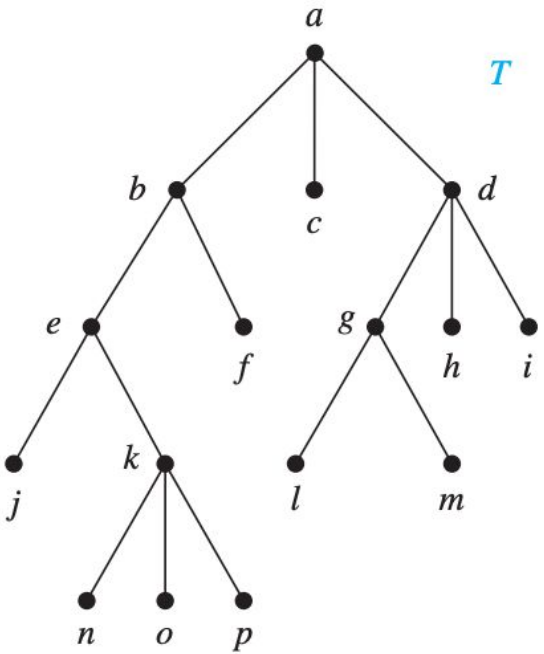


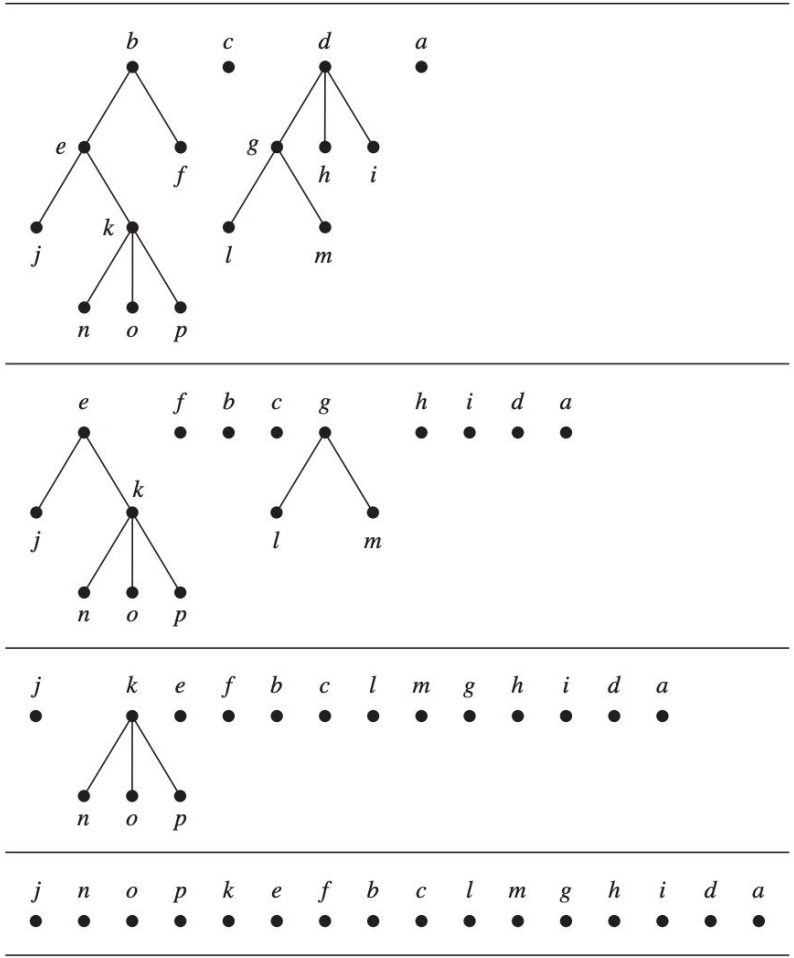
Postorder Traversal (20)





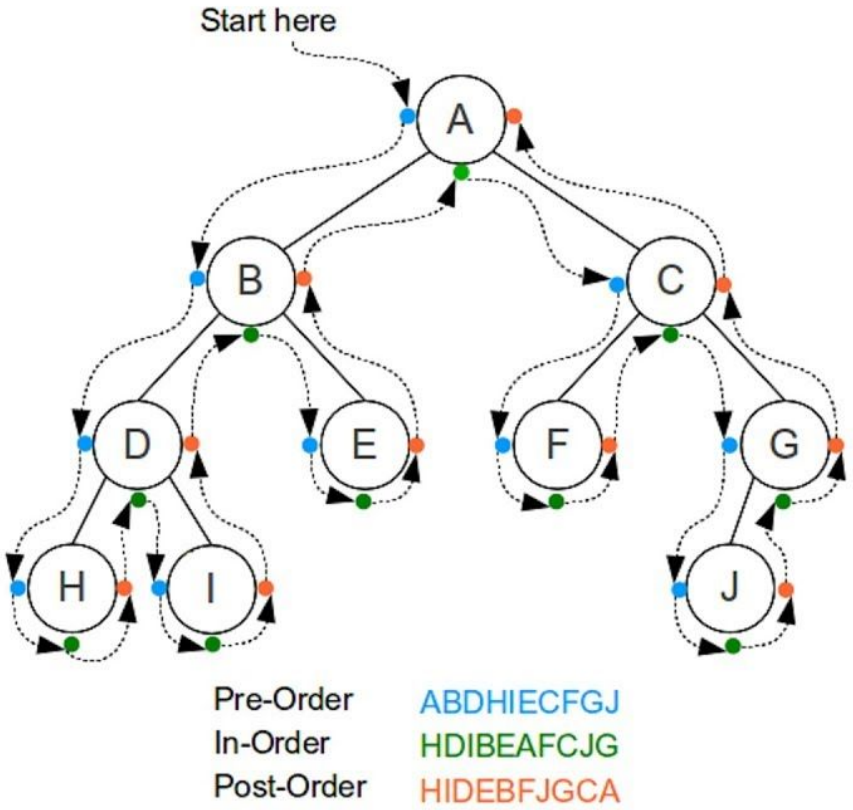
Postorder Traversal (21)





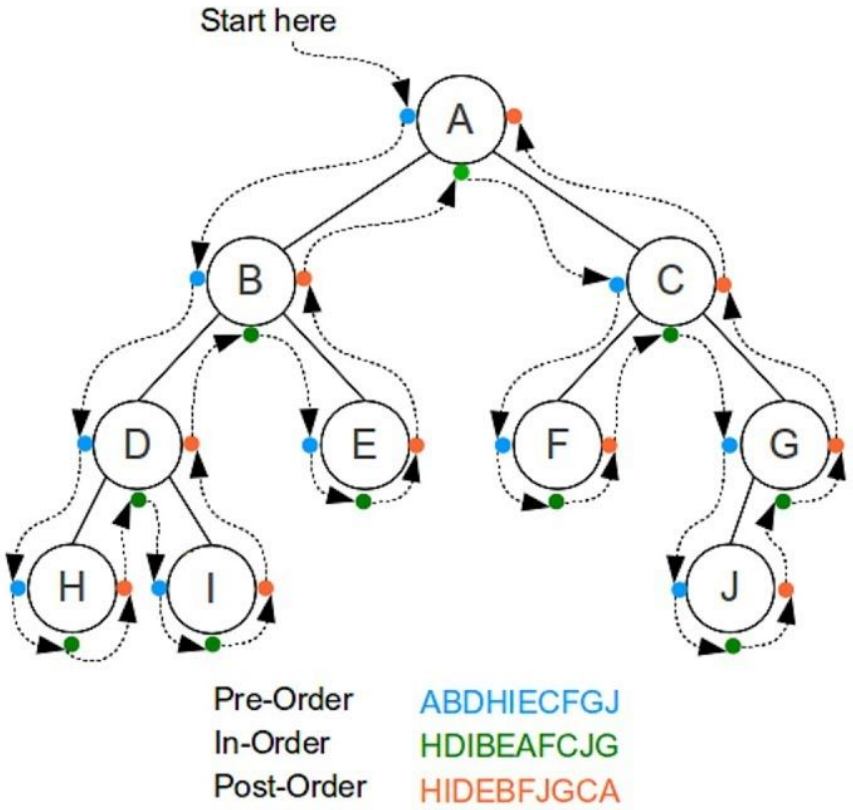
Dica & Análise!

- DFS (profundidade)
  - Preorder (raiz → esquerda → direita)
  - Inorder (esquerda → raiz → direita)
  - Postorder (esquerda → direita → raiz)



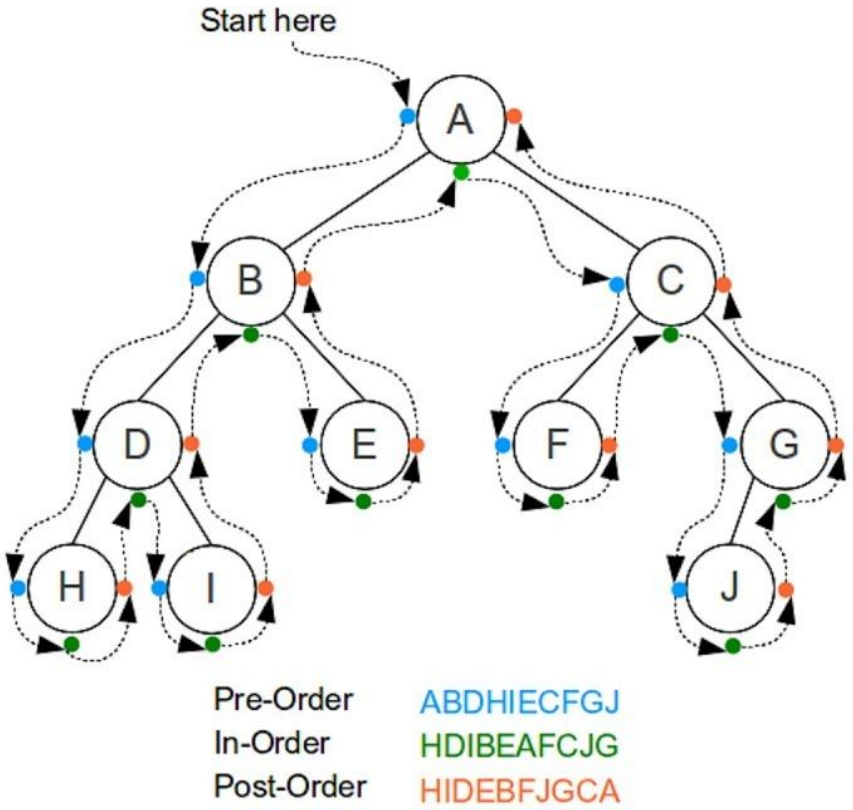
Dica & Análise!

- Complexidade de tempo
  - Todas as travessias visitam cada vértice exatamente uma vez.
    - $T(n) = O(n)$
      - onde n é número de nós da árvore.
  - Cada nó é processado uma única vez
  - Cada aresta é percorrida no máximo duas vezes (ida e volta, no caso recursivo)



Dica & Análise!

- Complexidade de espaço
  - Depende da **altura da árvore!**
- Usa pilha (explícita ou implícita da recursão)
  - Espaço:
    - $O(h)$ onde h é a altura da árvore
  - Casos:
    - Se Árvore balanceada: $h = O(\log n)$ $\Rightarrow$ $espaço = O(\log n)$
    - Se Árvore degenerada (tipo lista): $h = O(n)$ $\Rightarrow$ $espaço = O(n)$



Teste os algoritmos

- Link

Aplicações - *Travessals*

- Inorder, Preorder, and Postorder Traversals
  - Traversals são fundamentais na implementação de algoritmos
  - Árvores binárias representam expressões bem formadas
  - Tipos de notação gerados:
    - Preorder → Prefixa
    - Inorder → Infixa
    - Postorder → Posfixa
- Se a ordem não importa:
  - Qualquer travessia que visite cada vértice uma vez é válida
- Quando a ordem importa:
  - É necessário preservar relações específicas entre vértices
- Princípio geral:
  - Explorar vértices de interesse o mais cedo possível

Aplicações - *Travessals* (1)

- Quando usar *Preorder*
  - Ideal quando:
    - Vértices internos precisam ser visitados antes das folhas
  - Aplicações:
    - Cópia de árvores binárias
    - Estruturas hierárquicas
  - Curiosidade histórica:
    - Relacionado à ordem de sucessão em árvores genealógicas
- Quando usar *Postorder*
  - Ideal quando:
    - Folhas precisam ser processadas antes dos vértices internos
  - Aplicações:
    - Remoção de árvores
    - Processamento bottom-up
- Quando usar *Inorder*
  - Em árvores binárias de busca:
    - Visita os vértices em ordem crescente
  - Resultado:
    - Produz uma lista ordenada das chaves

Aplicações - *Travessals* (2)

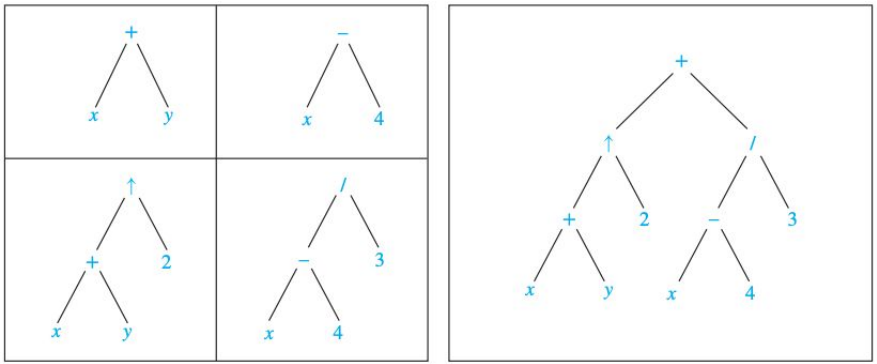
- Infix, Prefix, and Postfix Notation
  - Expressões complexas podem ser representadas por árvores enraizadas ordenadas
  - Exemplos de operações:
    - + (adição)
    - − (subtração)
    - × (multiplicação)
    - / (divisão)
    - ↑ (exponenciação)
  - Parênteses indicam a ordem das operações
- Representação em Árvores
  - Vértices internos → operadores
  - Folhas → variáveis ou números
  - Cada operador atua sobre:
    - Subárvore esquerda
    - Subárvore direita

Aplicações - *Travessals* (3)

- Problema: Encontrar a árvore enraizada ordenada que representa: $((x+y)\uparrow 2)+(x-4)/3$
  - Construção da Árvore - Passo a passo (bottom-up):
    1. Construir subárvore de (x + y)
    2. Aplicar exponenciação: (x + y) ↑ 2
    3. Construir subárvore de (x – 4)
    4. Construir divisão: (x – 4) / 3
    5. Combinar tudo com +

Aplicações - *Travessals* (4)

- Problema: Encontrar a árvore enraizada ordenada que representa: $((x+y)\uparrow 2)+(x-4)/3$
  - Estrutura da Árvore
    - Raiz: +
    - Subárvore esquerda: (x + y) ↑ 2
    - Subárvore direita: (x – 4) / 3
    - Regras:
      - Operadores → nós internos
      - Variáveis/constantes → folhas



Aplicações - *Travessals* (5)

- Problema: Encontrar a árvore enraizada ordenada que representa: $((x+y) \uparrow 2)+(x-4)/3$
  - *Inorder Traversal (Infix)*
    - Produz a expressão original: $x + y \uparrow 2 + x - 4 / 3$
      - **Qual é o problema?**
        - Pode gerar ambiguidades
        - Quando aplicamos *inorder traversal* (esquerda → raiz → direita), obtemos a expressão "natural", como escrevemos normalmente.
        - Não fica claro **qual operação acontece primeiro**
        - A ordem depende de precedência de operadores (↑, /, +, -)
        - Sempre que encontrar um operador em *inorder*, coloque parênteses ao redor da expressão formada por ele.
          - (esquerda operador direita)

Aplicações - *Travessals* (6)

- Problema: Encontrar a árvore enraizada ordenada que representa: $((x+y)\uparrow 2)+(x-4)/3$
  - *IPrefix Form Preorder (Polish Notation)*
    - Obtida por preorder traversal
    - Também chamada de:
      - Notação Polonesa
    - Características:
      - Não precisa de parênteses
      - Não é ambígua
      - Operador vem antes dos operandos

Aplicações - *Travessals* (7)

- *Infix vs Polish Notation*
  - Infix: É a forma mais comum, usada no dia a dia
    - O operador fica entre os operandos
      - x+y
      - $(x+y)\times z$
      - $a-b/c$
    - Pode ser ambígua
  - Polish:
    - O operador vem antes dos operandos
      - $+\ x\ y$ (equivale a $x + y$)
      - $*(+x\ y)z$ (equivale a $(x+y)\times z$
    - Não é ambígua
    - A ordem já está definida



JAN ŁUKASIEWICZ (1878–1956)

Jan Łukasiewicz nasceu em uma família de língua polonesa em Lviv. Na época, Lviv fazia parte da Áustria, mas hoje está na Ucrânia. Seu pai era capitão no exército austríaco. Łukasiewicz interessou-se por matemática ainda no ensino médio. Ele estudou matemática e filosofia na Universidade de Lviv, tanto na graduação quanto na pós-graduação. Após concluir seu doutorado, tornou-se professor assistente e, em 1911, foi nomeado professor titular. Quando a Universidade de Varsóvia foi reaberta como uma universidade polonesa em 1915, Łukasiewicz aceitou um convite para integrar o corpo docente. Em 1919, serviu como Ministro da Educação da Polônia. Ele retornou ao cargo de professor na Universidade de Varsóvia, onde permaneceu de 1920 a 1939, atuando duas vezes como reitor da universidade.

Łukasiewicz foi um dos cofundadores da influente Escola de Lógica de Varsóvia. Ele publicou sua obra inovadora *Elements of Mathematical Logic* em 1928. Com sua influência, a lógica matemática tornou-se uma disciplina obrigatória para estudantes de graduação em matemática e ciências na Polônia. Suas aulas eram consideradas excelentes, chegando a atrair estudantes das áreas de humanidades.

Łukasiewicz e sua esposa passaram por grande sofrimento durante a Segunda Guerra Mundial, o qual ele documentou em uma autobiografia publicada postumamente. Após a guerra, viveram no exílio na Bélgica. Felizmente, em 1949, ele recebeu uma posição na Academia Real Irlandesa, em Dublin.

Łukasiewicz trabalhou com lógica matemática ao longo de toda sua carreira. Seu trabalho em lógica de três valores foi uma contribuição importante para a área. No entanto, ele é mais conhecido nas comunidades de matemática e ciência da computação por sua introdução da notação sem parênteses, hoje chamada de **notação polonesa**.

Aplicações - *Travessals* (8)

- Problema: Qual é a forma **prefixa (*Polish notation*)** de: $((x+y) \uparrow 2)+(x-4)/3$
  - *R:* + ↑ + x y 2 / – x 4 3

Aplicações - *Travessals* (9)

- Postfix (*Reverse Polish Notation*)
  - Obtida por postorder traversal:
    - Esquerda → direita → raiz
- Problema: Qual é a forma postfix de: $((x+y)\uparrow 2)+(x-4)/3$
  - *R:* x y + 2 ↑ x 4 − 3 / +